

A MORK-Native Multi-Agent Architecture for Long-Horizon Mathematical Theorem Proving

**Technical design for OmegaClaw, PeTTa/MORK, ThreadKeeper,
Hyperon/PLN, reproducible computation, and Lean 4**

15 July 2026

Contents

Section	Subject
Extended Abstract	Basic idea, architectural commitments, and implementation path
1	Problem definition: theorem proving as persistent research search
2	Design goals, non-goals, and governing invariants
3	System overview and component responsibilities
4	The MORK-native proof metagraph
5	Atom schema, indexing, routes, and MORK/MM2 operations
6	Event journal, commit gate, snapshots, and crash recovery
7	Autonomous proof cycle and multi-agent scheduling
8	Context compilation and proof memory
9	Hyperon/PLN as a creative mathematical partner
10	Prompts and OmegaClaw skill architecture
11	ThreadKeeper worker integration and parallelism
12	Reproducible Python experiments and artifact provenance
13	Incremental Lean 4 verification
14	Worked case study: importing the N105-N109 proof effort
15	Reliability, security, observability, and evaluation
16	Implementation roadmap and extension points
Appendix A	MORK atom and event reference
Appendix B	Structured worker, context, event, and suggestion contracts
Appendix C	Prompt templates and skill specifications
Appendix D	Configuration example
References	Project documentation and research background

Document status. This is a standalone technical design and implementation blueprint, not a claim that every described component is already implemented upstream. The symbolic proof topology is placed natively in a MORK-backed PeTTa metagraph, while a revisioned event journal and content-addressed artifact store provide durability and provenance. Repository-specific observations reflect public project documentation accessed on 15 July 2026.

Extended Abstract

Long, speculative mathematical proofs require an AI system to do much more than generate the next paragraph. The system must preserve a changing research state over days or months; distinguish theorem, conjecture, computational evidence, failed mechanism, and useful bypass; retain abandoned options without continually replaying an enormous chat; manage several competing strategies; coordinate numerical and symbolic experiments; revisit old branches when a new lemma appears; and know when a striking result is genuine progress but does not prove the literal statement. A large chatbot context window can imitate part of this behavior, but it does not supply stable state semantics, graph operations, provenance, crash recovery, concurrency control, or independent verification. The human is therefore forced to act as a research director, repeatedly saying “take the next sharp move,” remembering the untried alternatives, and detecting silent changes in the theorem.

This document proposes a persistent, multi-agent proof-program architecture centered on OmegaClaw, PeTTa/MORK, ThreadKeeper, and Hyperon/PLN, with reproducible Python computation and later Lean 4 verification. Its central principle is that **the persistent proof project, not any LLM conversation, is the agent**. LLM calls, Hyperon runs, critics, experiment processes, and formalizers are replaceable workers. They receive a compiled view of the project and return structured proposals. The project survives provider changes, context truncation, worker crashes, and long pauses because its identity and history live outside the conversations.

A central architectural choice is the location of the proof graph. The symbolic topology of the proof project lives natively in a MORK-backed PeTTa space rather than in a conventional application database that must be mirrored into Hyperon. This is a natural fit for OmegaClaw because OmegaClaw already operates over PeTTa, while PeTTa can expose a MORK space that supports ordinary atomspace operations and MM2 transformations.¹ MORK’s PathMap representation, prefix-oriented indexing, space algebra, and rule transformations are useful for exact structural retrieval, reverse indexes, reachability, frontier views, dependency closures, and taint propagation.² The design nevertheless does **not** identify a MORK encoding path with a mathematical proof route. Semantic routes through the proof DAG are explicit derived objects, because different strategies may converge on the same state.

The persistent symbolic state is organized as three linked DAGs. The **search DAG** alternates OR-like proof states with AND-like moves that may create several required subgoals. It records alternatives, reductions, bypasses, generalizations, contradictions, and superseding strategies. The **justification DAG** records claims, exact dependency relations, support, criticism, contradiction, and formal verification. The **provenance DAG** connects attempts to contexts, prompts, model and persona versions, artifacts, experiments, critics, and Lean runs. These graphs share stable IDs and inhabit one proof metagraph, but their relations remain semantically distinct.

The MORK space contains compact typed atoms: node identities, edge relations, statuses, reverse indexes, route references, artifact hashes, and speculative hypotheses. Large theorem statements, proof prose, source code, datasets, papers, compiler logs, and binary outputs remain in a content-addressed local artifact store. A claim atom may therefore point to a statement blob by hash, and an attempt atom may point to a context manifest and result object. This keeps graph operations fast and makes every large object independently reproducible and backup-friendly.

MORK is the **semantic and query authority**, but the current public PeTTa/MORK wrapper is not treated as the sole durability or transaction mechanism. The public `mork_ffi` surface currently presents a small set of operations around a

¹ TrueAGI, *PeTTa* repository and *MORK spaces* documentation, accessed 15 July 2026. The documented MORK backend supports an `&mork` space, ordinary atom operations, matching, and `mm2-exec`. <https://github.com/trueagi-io/PeTTa> and <https://github.com/trueagi-io/PeTTa/wiki/MORK-spaces>

² TrueAGI, *MORK* wiki, especially *Data in MORK*, *Space Operations*, and the MM2 reachability tutorial, accessed 15 July 2026. <https://github.com/trueagi-io/MORK/wiki/Data-in-MORK>, <https://github.com/trueagi-io/MORK/wiki/Space-Operations>, and <https://github.com/trueagi-io/MORK/wiki/MM2-tutorial%3A-Reachability-P1>

process-global mutex-protected space.³ The design therefore places a deterministic proof-store service in front of MORK. Every committed mutation is first represented as a revisioned, append-only event bundle containing an expected revision, an optional lease fencing token, typed atom additions and removals, and artifact manifests. The event journal is the **durability authority**. A fresh MORK space can be rebuilt by replay, generated indexes can be recomputed, and the resulting digest can be compared with a checkpoint. The journal may be JSONL, a WAL-backed append-only event table, or another append-only store; it is not the conceptual home of the proof graph.

A deterministic **commit gate** is the only component allowed to mutate committed proof state. Explorers, critics, Hyperon processes, experimenters, and Lean formalizers write proposals. The gate checks schemas, stable IDs, theorem identity, legal status transitions, dependency consistency, artifact hashes, revision numbers, and active lease fencing tokens. New claims begin conservatively as conjectural, empirical, or provisional. Numerical agreement remains empirical unless the computation exhaustively proves the finite statement or produces an independently checkable certificate. A route-level substitute remains a bypass unless an explicit implication chain connects it to the literal target. If a claim is refuted or retracted, its dependent claims and closed states are tainted or reopened. Creative generation is thus separated from epistemic acceptance.

The most difficult memory problem is deciding what previous information belongs in the next worker prompt. The design treats this as **context compilation**, not informal summarization. Every attempt receives a theorem kernel, an active branch capsule, and retrieved ancillary context. The theorem kernel fixes the literal statement, definitions, domains, quantifier order, allowed background results, and constructivity or effectivity requirements. The branch capsule gives the state, route, selected move, established and provisional claims, open gaps, and latest failure, critic report, experiment, or Lean error. The ancillary context is assembled from four channels: exact MORK structural retrieval, uncertain PLN relevance, semantic retrieval over prose and literature notes, and deliberate diversity to counter tunnel vision. A context manifest records every source, score, status, inclusion reason, exclusion reason, version, token allocation, proof revision, lease token, and final packet hash.

OmegaClaw is the persistent coordinator and user-facing research manager. It normally invokes a small high-level surface such as `proof-create`, `proof-cycle`, `proof-status`, `proof-audit`, and `proof-export`. A `proof-cycle` atomically leases one frontier move, compiles context, dispatches a bounded persona, stores the complete result outside the digest channel, validates structured output, invokes a critic or tool worker when policy requires, commits accepted changes, updates MORK-derived frontier and taint views, and returns a compact milestone digest. Autonomy is implemented as a state machine and search policy: after repeated non-progress, generate five genuinely different mechanisms; queue all viable alternatives; backtrack after branch failure; and rebuild a root-level strategy portfolio after global stagnation.

ThreadKeeper supplies a useful bounded-worker pattern, but one theorem is not equated with one permanently alive child conversation. Current ThreadKeeper subagents are designed as synchronous, bounded, non-recursive dispatches that return a short digest.⁴ The mapping is therefore: one proof project equals one persistent workspace and MORK namespace; one selected move equals one bounded dispatch; one returned digest equals a status line and pointer to full attempt files. Two or three workers can initially share a proof-store process. They select moves through one atomic lease operation; a query followed by a separate “mark leased” update is forbidden because two workers could otherwise choose the same move. Expiring leases and monotonically increasing fencing tokens make late results safe to preserve but impossible to commit under an obsolete lease.

Hyperon has a larger role than retrieval or ranking. It is a **creative mathematical partner** intended to assume some of the guidance role that a human now plays. Because the proof topology already lives in MORK, Hyperon can reason directly over claims, techniques, obstacles, analogy mappings, failures, resource tradeoffs, and human guidance episodes. It writes structured hypotheses into an explicitly speculative layer: missing-lemma abductions, analogical transfers, concept blends, representation changes, obstruction inversions, counterexample constructions, stronger-but-cleaner lemmas, local-to-global mechanisms, and random/deterministic decompositions. Every hypothesis includes

³ Patham9, `mork ffi/src/lib.rs`, public source accessed 15 July 2026. The present wrapper uses a process-global Space protected by a mutex and exposes a compact operation surface. This observation is about the reviewed public source and may change in later revisions. https://github.com/patham9/mork_ffi/blob/master/src/lib.rs

⁴ ThreadKeeper, *Subagent Dispatch - Design*, accessed 15 July 2026. <https://github.com/hlgreenblatt/ThreadKeeper/blob/threadkeeper/docs/subagent-design.md>

source atoms, prerequisites, expected consequences, novelty and abductive-support annotations, an early falsification test, and a review condition. An LLM elaborates selected hypotheses into explicit mathematics. Critics, experiments, Lean, and the commit gate determine what becomes accepted. PLN truth values and analogy scores never map directly to theorem status.

Parallel search resembles a chess engine only in part. Mathematical moves are dynamically generated, a single move may create several mandatory subgoals, and transpositions are common. The first scheduler should therefore be an auditable best-first AND/OR search using MORK to construct the eligible candidate set and Python or PLN to score it by promise, novelty, expected information gain, verification value, cost, and repeated-failure risk. Later, after enough empirical attempt data exists, the scheduler can add PUCT, progressive widening, proof-number signals, transposition statistics, and model/persona diversity. Objective signals—subgoals closed, critics survived, claims formally verified, contradictions discovered, branches pruned—are preferred to an LLM’s self-reported confidence.

Python experiments and Lean 4 verification operate at attempt and claim granularity. Each experiment begins with a specification stating the mathematical question, competing hypotheses, domain, seed policy, expected outcomes, and inferential limits. Code, commands, environment, inputs, stdout, stderr, results, certificates, and hashes are retained. Lean is introduced incrementally: each stable claim receives a statement-alignment record, stable declaration name, pinned toolchain and library revision, small source file, and classified checker result. Only the exact declaration and dependency versions accepted by Lean become `lean-verified`.

The supplied proof interaction is a useful realistic fixture. Its N105–N109 progression includes conceptual compression of many moment calculations into a general theorem; a literal target left open while a deterministic algebraic shield supplies a route-level substitute; a spectral shield that changes the entropy mechanism; a Hall-expander shield that strengthens algebraic immunity; attached Python verifiers, PDFs, LaTeX, and logs; and a final high-degree residual target that remains active. A transcript can narrate the sequence. A MORK proof metagraph can represent it precisely through relations such as `supersedes`, `bypasses`, `strengthens-route`, `depends-on`, `verified-by-experiment`, `exposes-barrier`, and `leaves-open`.

The resulting architecture is intentionally simple at its safety-critical core: one proof-store owner per active theorem, a MORK semantic space, an append-only event journal, a content-addressed artifact store, narrow OmegaClaw skills, bounded ThreadKeeper workers, and explicit commit rules. It is maintainable enough to build now, while leaving clean extension points for richer MM2 transformations, PLN creativity, learned scheduling, asynchronous workers, formal proof, and larger-scale multi-searcher theorem exploration.

1. Problem Definition and Design Goals

1.1 The task is research management as well as theorem proving

A long speculative proof is not a single completion problem. It is a changing research program. The program contains a literal target theorem, intermediate lemmas, competing strategies, failed reductions, partial computations, suspected obstructions, stronger substitute statements, and external evidence. Some objects are crisp mathematical claims; others are search hypotheses about what kind of argument might work. The system must preserve both without confusing them.

A chatbot transcript conflates several distinct things:

- the actual mathematical proof state;
- the model’s latest narrative summary of that state;
- the conversational history needed to reconstruct earlier decisions;
- the list of promising moves that were mentioned but never tried;
- tool outputs and local files;
- the model’s confidence, which may not correlate with correctness.

The architecture in this document separates these concerns. It regards natural-language dialogue as an interface and an immutable record, not as the state schema of the proof.

1.2 Characteristic failure modes of chat-only proof work

The design is driven by recurrent failure modes observed in long LLM proof sessions.

Context attrition. Earlier definitions, subtle quantifier choices, or the reason a route was abandoned disappear from the active context. The model reconstructs them approximately and introduces theorem drift.

Unrecorded alternatives. A response may list three good possibilities and pursue one. The other two remain only in prose and are rarely revisited.

Status collapse. A plausible lemma, a numerically supported pattern, a critic-accepted argument, and a formally verified theorem may all be described with similar language. Later steps use the wrong epistemic level.

Narrative backtracking. When a branch fails, the model has no explicit search state. It may repeat the same idea in different words or return to the root without preserving what was learned.

Substitute-target confusion. A stronger-looking construction or barrier theorem may substantially improve the route while leaving the literal theorem open. Conversational momentum encourages calling the route “complete.”

Artifact detachment. Python code, results, PDFs, logs, and literature notes are generated, but their relation to a particular branch and claim is not stable.

Concurrency without coordination. Several agents can be asked to work in parallel, but without shared leases and a common claim ledger they duplicate work, overwrite assumptions, or return incomparable reports.

Human as scheduler. The human repeatedly decides that it is time to continue, brainstorm, backtrack, challenge a claim, run a computation, or try a radically different mechanism. Much of this scheduling work is mechanical enough to externalize.

1.3 Functional requirements

The first implementation should satisfy the following requirements.

1. It must resume a proof from local files without relying on provider-side chat history.
2. It must preserve every attempt and every artifact under stable identifiers.
3. It must represent alternatives and mandatory subgoals explicitly.
4. It must distinguish claim status from heuristic promise.
5. It must compile a bounded, relevant context for each new LLM call.
6. It must autonomously continue, backtrack, branch, and audit under configured budgets.
7. It must allow two or three workers to share one frontier safely.
8. It must permit Python computation now and Lean 4 verification later.
9. It must allow Hyperon/PLN to influence both retrieval and creative strategy generation.
10. It must export deterministic, Git-friendly snapshots and remain understandable to a human maintainer.

1.4 Non-goals of the initial implementation

The initial implementation does not need to make the MORK runtime itself the sole durability layer, implement a fully general mathematical ontology, run dozens of asynchronous workers, perform end-to-end Lean formalization, or learn an accurate value function for every branch. It also should not attempt to replace mathematical judgment with a scalar confidence score. The architecture is staged precisely so that a useful persistent proof agent exists before the most ambitious components are added.

1.5 Design vocabulary

A **proof** is one persistent project anchored by a literal theorem specification. A **state** is a snapshot of assumptions, accepted dependencies, provisional dependencies, and open goals. A **move** is a proposed direction from a state. An **attempt** is one bounded execution of one move by one worker. A **claim** is a reusable mathematical proposition. An **artifact** is a file or external object generated or used by an attempt. A **suggestion** is a heuristic proposal that may become a move but is not itself a claim. A **verification** is an independent status-changing evaluation by a critic, exhaustive computation, or formal prover.

2. Architectural Principles

2.1 The persistent proof project is the agent

The most important principle is that continuity belongs to the proof workspace. An LLM call can be discarded and retried. A ThreadKeeper persona can be swapped for another model. OmegaClaw can be restarted. The proof remains because its exact state is on disk.

This inversion has several practical consequences:

- worker prompts must be self-contained;
- workers return structured proposals rather than editing the graph freely;
- every summary must cite source IDs;
- the proof engine can reproduce the context of any attempt;
- a new model can continue an old proof without reading the entire transcript;
- provider-side conversation features become optional conveniences rather than dependencies.

2.2 Separate generative cognition from authoritative state mutation

LLMs and Hyperon are most valuable when they can generate hypotheses, analogies, reductions, and candidate arguments. They are least reliable when asked to silently maintain a complicated transactional state. The design therefore gives generative components broad freedom to propose and a narrow, typed interface through which proposals can become durable state.

The proof engine is deliberately boring. It validates identifiers, dependencies, status transitions, leases, files, and theorem hashes. This separation makes the system easier to debug: a bad mathematical idea is stored as a failed attempt, while a bad committed-state mutation is rejected as a protocol error.

2.3 Use four memories, not two

Two obvious memory layers are a complete history and a structured option tree. A robust system needs four coupled memories, all addressable through the proof metagraph.

1. **Immutable attempt history.** Every context packet, prompt, response, tool call, critic report, experiment, and verifier result.
2. **Search topology.** States, moves, alternatives, required subgoals, bypasses, contradictions, backtracking, and convergent routes.
3. **Claim ledger.** Reusable propositions, exact statement blobs, dependencies, provenance, and verification state.
4. **Artifact memory.** Code, data, papers, LaTeX, Lean files, logs, manifests, and content hashes.

The architecture represents the symbolic parts of all four memories in a MORK-backed PeTTa space. The artifact payloads remain in a content-addressed filesystem, while a revisioned event journal makes all committed metagraph mutations durable and replayable. This division lets Hyperon reason directly over the same graph used by the scheduler and context compiler without putting multi-megabyte blobs into the atomspace.

2.4 The graph is an AND/OR DAG

A tree is a useful visual projection, but the persistent object is an AND/OR DAG embedded in a larger metagraph.

- A **state** is normally an OR point: one successful outgoing strategy may close it.
- A **move** is normally an AND point: if it creates several subgoals, all of them must close.
- A claim discovered on one branch can support several other states.
- Distinct strategies may converge on the same state.
- A theorem may be bypassed by a stronger route-level result without being literally proved.

Stable node IDs are independent of display addresses. A label such as `root/strategy-3/lemma-2` is a route, not identity. An attempt records both its state and route, because route-specific assumptions and context can differ even when the terminal state is shared.

MORK's PathMap paths should not be confused with these semantic routes. The PathMap indexes encoded expressions by shared prefixes. Semantic reachability and route projection are represented by explicit atoms and MM2 transformations.

2.5 Exact theorem identity is a first-class invariant

The theorem specification should be normalized and hashed. It includes the statement, definitions, quantifier order, domains, asymptotic regime, admissible background assumptions, constructivity requirements, and any intended uniformity. A proof-complete candidate is checked against this kernel. This prevents a common failure in which an agent proves a nearby statement and treats it as the original target.

2.6 Raw records are immutable; summaries are derived

Attempts, prompts, responses, tool outputs, and events are append-only. Branch summaries and global summaries may be regenerated as better compression methods become available, but they must record the source IDs and versions from which they were derived. No failed attempt is erased. The history is a scientific lab notebook, not a polished retrospective.

2.7 Epistemic status is discrete and provenance-bearing

Suggested claim statuses are:

Status	Meaning
conjectural	proposed, not independently supported
supported	a coherent argument or empirical evidence exists, but gaps may remain
critic_accepted	an independent critic found no local defect under the recorded assumptions
lean_verified	the exact formal declaration was checked by a pinned Lean environment
refuted	a counterexample or decisive contradiction was found
retracted	withdrawn because a dependency or interpretation failed

The status is not a floating “confidence.” It is a protocol state tied to evidence. Hyperon's probabilistic truth values are useful for heuristic reasoning, but they do not replace this ledger.

2.8 Start with maintainable infrastructure

The first implementation should be conservative:

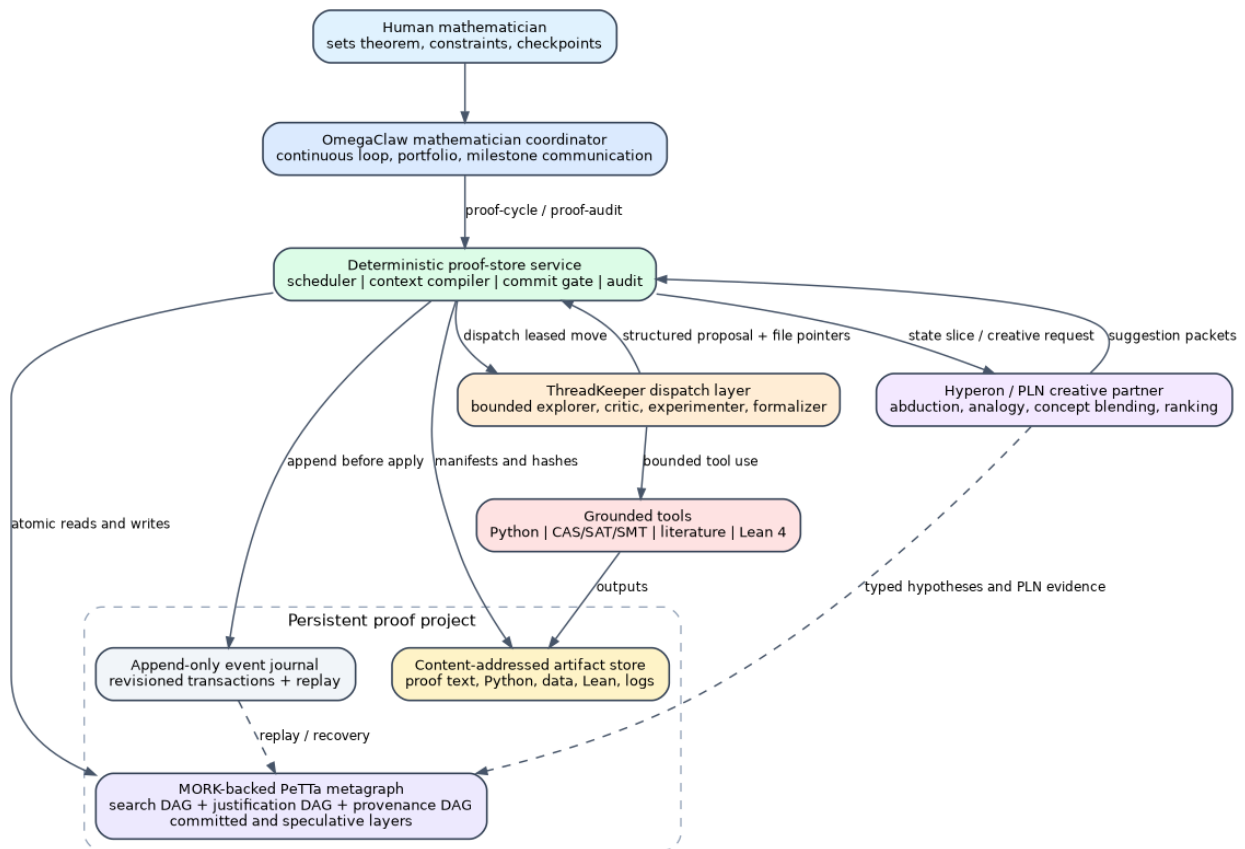
- one proof-store process per active theorem;
- a MORK-backed PeTTa space as semantic/query authority;

- an append-only JSONL journal or the event journal WAL-backed event log as durability authority;
- content-addressed files for prose and artifacts;
- stable compact IDs, schemas, and atomic leases;
- one explorer and one critic before broad parallelism;
- reproducible experiments before autonomous tool proliferation;
- explicit adapters for OmegaClaw, ThreadKeeper, Hyperon, and Lean.

This is not a retreat from neural-symbolic ambition. It is what makes the ambition testable. MORK, PLN, creative rules, and learned scheduling can become progressively more capable without making proof history, transaction semantics, or theorem identity depend on an opaque agent conversation.

3. End-to-End System Architecture

3.1 Component overview



MORK-native end-to-end architecture

The architecture has seven cooperating layers.

Human mathematician. Defines or imports the theorem kernel, resolves semantic ambiguity, reviews durable milestones, and can seed or veto strategies. Human interventions are guidance events, not proof evidence.

OmegaClaw mathematician coordinator. Owns the user relationship and portfolio-level control loop. OmegaClaw’s published loop assembles each model call from its prompt, skill catalogue, previous skill results, history, and time.⁵ The proof integration exploits that loop while keeping detailed mathematical state external and authoritative.

Deterministic proof-store service. Owns revisions, leases, context compilation, schema validation, commit, replay, audit, export, and status propagation. It is the sole writer of committed proof atoms.

MORK-backed PeTTa proof metagraph. Stores the symbolic topology of search, justification, provenance, speculative hypotheses, reverse indexes, and derived route/frontier views. PeTTa’s MORK space supports ordinary atomspace operations together with MM2 execution.⁶

Event journal and artifact store. The journal durably records every committed atom batch. The artifact store retains large proof text, code, data, papers, contexts, responses, and verifier outputs by content hash.

ThreadKeeper worker layer. Runs bounded explorers, critics, experimenters, analogy specialists, and formalizers. Full output is written into attempt directories; the returned digest is only a pointer and status line.

Hyperon/PLN creative partner and tools. Hyperon retrieves, evaluates, and creates speculative mathematical hypotheses. Python, literature search, symbolic systems, SAT/SMT tools, and later Lean 4 supply external checks and artifacts.

3.2 Authority boundaries

The design deliberately separates three authorities.

MORK metagraph	semantic and query authority
append-only journal	durability and replay authority
artifact store	byte-level authority for large objects

The committed MORK state and journal should agree after every successful transaction. Generated indexes and route views are rebuildable. A hash manifest connects symbolic references to immutable artifacts.

The current public `mork_ffi` wrapper is small and appears to expose one process-global mutex-protected space with add, remove, match, dump, and MM2 execution operations.⁷ That is sufficient for a prototype semantic backend but not yet the full transaction and persistence surface wanted by this proof system. The proof-store service supplies revision checks, batch semantics, fencing tokens, journal append, replay, and snapshots without requiring those concerns to be encoded in prompts.

3.3 Why OmegaClaw coordinates but does not own proof memory

OmegaClaw is a continuous neural-symbolic agent with an extensible skill catalogue.⁸ It is a good owner of high-level pacing and human communication. It should not store the whole proof graph in its ordinary history. The coordinator remains thin:

1. read compact status and milestone digests;
2. invoke proof-cycle within budget;
3. invoke proof-audit on schedule or before a completion claim;
4. request Hyperon creative expansion after stagnation;
5. surface milestones, contradictions, theorem ambiguities, and resource decisions.

⁵ ASI Alliance, *OmegaClaw Internals: Execution Loop*, project documentation, accessed 15 July 2026. <https://raw.githubusercontent.com/asi-alliance/OmegaClaw-Core/main/docs/reference-internals-loop.md>

⁶ TrueAGI, *PeTTa* repository and *MORK spaces* documentation, accessed 15 July 2026. The documented MORK backend supports an `&mork` space, ordinary atom operations, matching, and `mm2-exec`. <https://github.com/trueagi-io/PeTTa> and <https://github.com/trueagi-io/PeTTa/wiki/MORK-spaces>

⁷ Patham9, `mork_ffi/src/lib.rs`, public source accessed 15 July 2026. The present wrapper uses a process-global `Space` protected by a mutex and exposes a compact operation surface. This observation is about the reviewed public source and may change in later revisions. https://github.com/patham9/mork_ffi/blob/master/src/lib.rs

⁸ ASI Alliance, *OmegaClaw-Core*, GitHub repository, accessed 14 July 2026. <https://github.com/asi-alliance/OmegaClaw-Core>

Provider context may disappear; the project does not.

3.4 Why ThreadKeeper workers are ephemeral

A theorem may run for weeks, but one worker should receive a bounded, sharply specified move. Current ThreadKeeper documentation describes a synchronous bounded subagent loop, no subagent-to-subagent recursion, and a short returned digest.⁹ The recommended mapping is:

```
one theorem project -> one persistent workspace and MORK namespace
one proof branch   -> persistent state/move/claim atoms
one proof attempt  -> one bounded ThreadKeeper dispatch
one digest         -> attempt ID, disposition, and result path
```

The complete mathematical answer lives in `attempts/<id>/result.json` and `proof-text.md`, not in the digest.

3.5 The proof-store service as anti-corruption layer

Framework APIs and model providers will change. The proof model should not. The service exposes a small stable API:

```
proof-create(spec_path) -> proof_id
proof-cycle(proof_id) -> digest
proof-status(proof_id) -> digest
proof-lease-next(proof_id, worker, ttl) -> lease
proof-commit-result(proof_id, result_path) -> commit
proof-audit(proof_id) -> report
proof-export(proof_id, destination) -> snapshot
```

Internally it can call PeTTa/MORK, ThreadKeeper, PLN, Python, Lean, or a vector index through adapters. Upstream changes remain localized.

3.6 One proof-store owner per theorem

The simplest safe concurrency model is one service process per active theorem. It owns that proof's MORK space, journal revision, leases, and maintenance rules. OmegaClaw and worker processes communicate through a narrow local API or Unix socket. A separate global mathematical knowledge space may hold reusable concepts and verified lemmas, but a cross-proof import becomes an explicit event with source version and provenance.

This model naturally shards contention and avoids multiple processes mutating a process-global `&mork` directly.

4. Persistent Proof State and MORK Data Model

4.1 Workspace layout

A practical local workspace is:

```
proof-workspace/
  proofs/<proof-id>/
    journal/
      events.jsonl
      checkpoint.json
      snapshots/
    theorem/
      theorem-spec.yaml
      theorem-kernel.md
  attempts/<attempt-id>/
    worker-request.json
    context.md
```

⁹ ThreadKeeper, *Subagent Dispatch - Design*, accessed 15 July 2026. <https://github.com/hlgreenblatt/ThreadKeeper/blob/threadkeeper/docs/subagent-design.md>

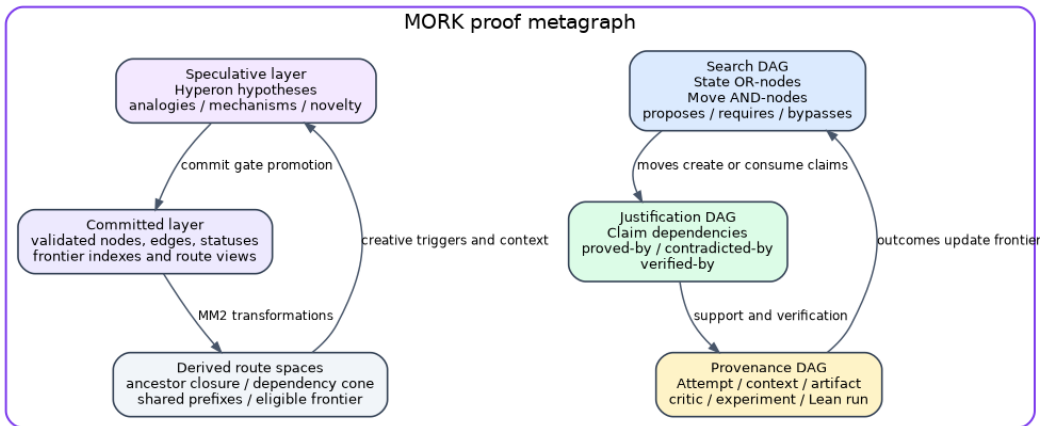
```

context-manifest.json
prompt.md
response.md
result.json
proof-text.md
artifacts/
verification/
summaries/
global.md
states/
claims/
exports/
objects/sha256/<prefix>/<digest>
manifests/<artifact-id>.json
prompts/
skills/
schemas/

```

The MORK space is opened by the proof-store service and may be rebuilt from the journal. The sidecar files make the project inspectable and Git friendly. Large objects can be backed up through Git LFS or another object store while retaining the same content identity.

4.2 Three linked DAGs in one metagraph



Search, justification, provenance, and speculative layers

Search DAG

States and moves encode the evolving search. Core relations include proposes, requires, alternative, generalizes, bypasses, supersedes, contradicts, reformulates, and formalizes.

Justification DAG

Claims are exact propositions with status and dependencies. Relations include depends-on, supported-by, proved-by, contradicted-by, verified-by, and invalidates.

Provenance DAG

Attempts link to state, move, route, context, prompt, response, worker, model, artifacts, experiments, critics, and formalization runs.

Speculative layer

Hyperon hypotheses and uncriticized move proposals are explicit speculative atoms. They can affect retrieval and scheduling but cannot be used as established premises.

4.3 Ground-prefix atom schema

MORK's documentation emphasizes data layout and efficient prefix matching.¹⁰ Put stable ground namespace fields first and variable positions later:

```
(proof p17 layer committed state s101)
(proof p17 layer committed move m204)
(proof p17 out s101 proposes m204 e901)
(proof p17 in m204 proposed-by s101 e901)
(proof p17 out m204 requires s102 e902)
(proof p17 claim-status c308 provisional)
(proof p17 depends-on c310 c308)
(proof p17 produced-claim a411 c308)
(proof p17 attempt-via-route a411 r52)
(proof p17 artifact-blob art92 b3_4VQ7ZJ6)
```

Forward and reverse edge atoms are intentionally duplicated as generated indexes. They are projections from one semantic edge ID, not independent mathematical assertions.

4.4 Stable identity and route identity

State `s101` is a stable state. Route `r52` is one way of reaching it. Another strategy may reach the same state through `r87`. Attempts record both:

```
(proof p17 attempt-at-state a411 s101)
(proof p17 attempt-via-route a411 r52)
```

This avoids duplicating shared lemmas while preserving route-specific assumptions and context. Display addresses may be regenerated and must not be database keys.

4.5 Core node types

Proof/theorem. Literal theorem kernel, active revision, root state, and project policy.

State. Current goal, assumptions, established and provisional claim references, open/closed/tainted status, and branch summary.

Move. A proposed operation, technique family, target state, required subgoals, score components, cost estimate, and status.

Attempt. One execution of one move under one route, revision, context packet, worker persona, and lease.

Claim. Exact statement blob, dependencies, status, source attempts, critics, experiments, Lean runs, and taint sources.

Artifact. Content hash, media type, semantic role, producing attempt, and manifest.

Hypothesis. A speculative Hyperon suggestion with kind, source concepts, prerequisites, consequences, scores, and falsification test.

4.6 Claim status and truth values

Protocol statuses are discrete:

```
conjectural
empirical
provisional
critic-accepted
lean-verified
refuted
```

¹⁰ TrueAGI, MORK wiki, especially *Data in MORK*, *Space Operations*, and the MM2 reachability tutorial, accessed 15 July 2026. <https://github.com/trueagi-io/MORK/wiki/Data-in-MORK>, <https://github.com/trueagi-io/MORK/wiki/Space-Operations>, and <https://github.com/trueagi-io/MORK/wiki/MM2-tutorial%3A-Reachability-P1>

retracted
stale

PLN strength/confidence, model confidence, novelty, and scheduler value are annotations. They cannot substitute for claim status. A high PLN truth value may cause a hypothesis to be explored sooner, but not to be used as an established premise.

4.7 Derived spaces and path algebra

MORK space operations are useful for candidate-set algebra:

```
EligibleFrontier = (OpenMoves union ReopenedMoves)
                  subtract LeasedMoves
                  subtract RefutedMoves
                  subtract DominatedMoves
                  subtract ExhaustedMoves
```

Other derived views include dependency closure, ancestor routes, shared route prefixes, tainted claims, sibling failures, and context candidates. MM2 transformations can materialize reverse indexes, reachability, generations, route projections, and bounded taint propagation. Numeric scoring may be computed in Python or PLN while exact set construction remains symbolic.

A MORK encoding path is not itself a semantic proof route. The latter is constructed from explicit search-edge atoms.

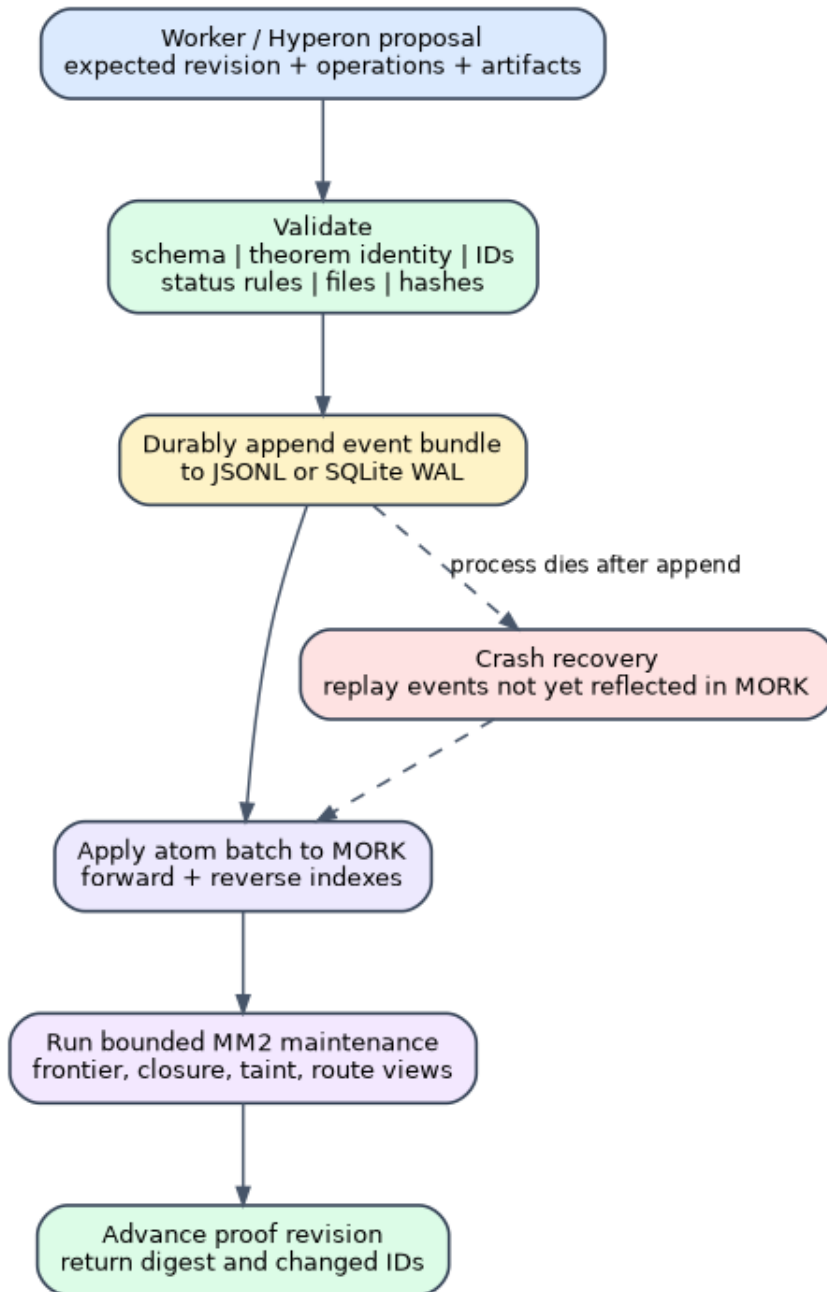
4.8 Artifact references and symbol constraints

Keep atom arities regular and symbols compact. Current MORK encoding documentation describes practical limits in its tag layout for symbol lengths, arities, and variable references.¹¹ Do not insert enormous statements, raw filesystem paths, or a multi-kilobyte JSON document as a single atom symbol. Store a compact ID or base32 digest and resolve it through a manifest.

4.9 Event sourcing and authoritative mutation

Every committed change is an event bundle with an expected revision and atom operation batch. The event is sufficient to rebuild the committed MORK state. Generated views can be recomputed from committed atoms and versioned maintenance rules.

¹¹ TrueAGI, MORK wiki, especially *Data in MORK*, *Space Operations*, and the MM2 reachability tutorial, accessed 15 July 2026. <https://github.com/trueagi-io/MORK/wiki/Data-in-MORK>, <https://github.com/trueagi-io/MORK/wiki/Space-Operations>, and <https://github.com/trueagi-io/MORK/wiki/MM2-tutorial%3A-Reachability-P1>



Journal-first commit and replay protocol

The transaction sequence is:

1. validate worker result, theorem identity, statuses, and artifacts;
2. acquire the proof lock and recheck revision and lease token;
3. append and flush the event;
4. apply the atom batch to MORK;
5. run bounded maintenance transformations;
6. advance the revision and write a digest checkpoint.

If the process dies after journal append but before MORK application, replay completes the transaction.

4.10 Taint propagation and reopening

When a claim is refuted, retracted, or changed:

1. mark direct dependents tainted;
2. propagate taint through the justification DAG;
3. mark downstream Lean verification stale when dependency versions no longer match;
4. reopen closed states that depended on the claim;
5. preserve historical attempts and old statuses for audit.

MM2 may derive the affected cone, but the commit gate records the protocol status transitions.

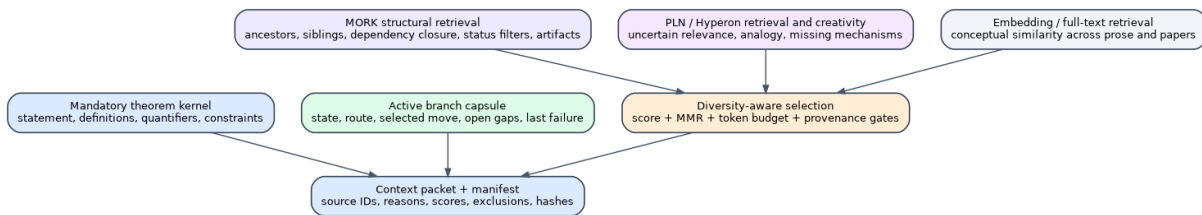
5. Context Compilation: Solving the Difficult Memory Problem

5.1 Why context selection is the central systems problem

Once proof state is externalized, the next worker does not automatically inherit the whole conversational past. This is beneficial because the past may contain hundreds of pages of redundant or obsolete material, but it creates the hard question identified in the original proposal: what information from earlier work is possibly useful for the next proof step?

The wrong answers are at opposite extremes. Supplying the complete history exhausts the context window, buries the current obstruction, and causes the model to imitate stale reasoning. Supplying only the latest branch summary loses definitions, dependency provenance, sibling failures, and distant lemmas that have become newly relevant. Free-form summarization is not enough because a summary generator can omit exactly the detail needed by a later argument and gives no record of why a particular item was included.

The proposed solution is a compiler-like pipeline. Inputs are typed records in the proof workspace; output is a bounded context packet with a manifest. Like a conventional compiler, it has mandatory inputs, candidate generation, scoring, optimization under a resource budget, and a reproducible build product.



Context compilation

Context compilation pipeline.

5.2 The three mandatory context blocks

Every mathematical worker receives three top-level blocks.

Theorem kernel

The theorem kernel is always included verbatim or by a versioned canonical rendering. It contains:

- literal theorem statement
- definitions and notation
- quantifier order and domains
- asymptotic regime and uniformity requirements
- allowed background theorems

constructivity, effectivity, or complexity requirements
 explicit nearby statements that do not suffice
 hash of the normalized theorem specification

The last two items are important. A worker must know not only the target, but also common substitutes that should not be mistaken for it. For a complex research program, the kernel may also name route-level objectives while marking them as distinct from the literal theorem.

Active branch capsule

The branch capsule describes the exact local proof state:

state ID and display path
 selected move ID and move relation
 assumptions currently in force
 established claim IDs with short statements
 provisional claim IDs, visibly marked as provisional
 open goals and their AND/OR semantics
 most recent attempt and its disposition
 latest critic, experiment, or Lean feedback
 branch-local taboo list: mechanisms already falsified here

The capsule should be compact enough to include on every retry. It is generated deterministically from the state graph and claim ledger rather than copied from an old narrative summary.

Retrieved ancillary context

Ancillary context contains information that is not mandatory for correctness but may materially improve the next move. Sources include:

- ancestor-state summaries;
- sibling branches that attacked the same subgoal;
- prior failures with a matching obstruction;
- reusable claims in the dependency neighborhood;
- relevant experiment interpretations and formalization errors;
- literature notes attached to the same technique or object;
- human guidance episodes in analogous states;
- Hyperon-derived analogies or abductive bridge hypotheses;
- one deliberately dissimilar strategy when the branch is at risk of tunnel vision.

The packet clearly identifies every item by source ID and status. A worker may quote a source ID in its result, allowing the system to measure which retrieved items were actually useful.

5.3 Candidate generation

Candidate context comes from four coordinated channels.

MORK structural retrieval. Query dependency and support closures, route ancestors, failed siblings, shared concepts, contradictions, bypasses, artifacts, and previous attempts on the same state. This channel is exact and status aware.

Hyperon/PLN retrieval and generation. Use uncertain support, analogy, mechanism transfer, and abductive missing-lemma relations to surface non-obvious material. Hyperon may also create a small number of new speculative hypotheses targeted at the current obstruction.

Semantic retrieval. Search proof prose, literature notes, code summaries, and external-source artifacts using embeddings or lexical methods. Each retrieved chunk retains its owning artifact and revision.

Deliberate diversity. When a branch is stale, add one or two structurally dissimilar strategy records even if their nearest-neighbor score is lower. This is a controlled antidote to tunnel vision.

Candidate generation is therefore not “search the transcript.” It is a join across exact graph structure, uncertain symbolic inference, and textual memory.

5.4 Baseline scoring and diversity

A transparent baseline score is preferable to an opaque learned ranker at the beginning. One workable formula is:

$$S_i = 4 D_i + 3 A_i + 2.5 E_i + 2 V_i + 1.5 F_i + 1.3 P_i + N_i - 1.5 R_i - \lambda C_i.$$

Here D_i measures dependency relevance, A_i ancestry or route proximity, E_i semantic similarity, V_i verification status, F_i a matching failure mode, P_i PLN support or relevance, N_i novelty, R_i redundancy, and C_i token cost.

The exact coefficients are configuration, not doctrine. Their value is that every inclusion is explainable. Later, learned or PLN-derived scores can be blended in without removing the baseline features.

Selection should use a diversity-aware method such as maximal marginal relevance. Otherwise the top-ranked set often contains several paraphrases of one idea. The compiler reserves quotas for different evidence classes, for example:

```
40% dependency and ancestor material
20% failures and critic feedback
15% experiments or formalization feedback
15% analogies and creative suggestions
10% deliberately dissimilar material
```

These percentages can change with the worker role. A critic receives more dependency and failure material; an analogy explorer receives more cross-branch and literature material.

5.5 Hyperon-assisted relevance

Because the proof graph already inhabits MORK, Hyperon does not need a lossy mirror before reasoning. It can attach uncertain relevance and mechanism-support relations directly to proof IDs and concept atoms. Examples include:

```
claim C is structurally analogous to the active obstruction
mechanism M succeeded on a branch with the same failure signature
hypothesis H would bridge two dependency components
artifact A contains a counterexample generator relevant to this state
```

A PLN score is one feature among several. It should be accompanied by source atoms and an evidence stamp. The compiler can prefer verified or critic-accepted material while still including a highly novel speculative hypothesis when its falsification cost is low.

The resulting context pipeline is:

```
MORK exact structure
+ PLN uncertain relevance and creative hypotheses
+ semantic retrieval
+ diversity and token allocation
= context packet plus manifest
```

5.6 Token budgeting

The compiler allocates a fixed input budget before dispatch. A practical default for a large-context model is:

Block	Fraction of input budget
Theorem kernel and protocol	15%
Active branch capsule	25%
Established and provisional claims	20%
Retrieved ancillary context	30%
Output schema and tool instructions	10%

The engine should not fill the context merely because space is available. Extra material can decrease performance. A branch with one crisp local obstruction may receive a much smaller packet than a root-level strategy review.

Long records are represented by a hierarchy of views:

one-line index entry
 short claim or attempt capsule
 branch summary
 full source record
 artifact itself

The compiler chooses the smallest view sufficient for the role. A worker can request an additional source by ID during its bounded tool loop, and that access is logged.

5.7 Context manifest

Every compiled packet has a machine-readable manifest:

```
{
  "context_id": "ctx-01J...",
  "proof_id": "proof-...",
  "state_id": "state-...",
  "move_id": "move-...",
  "theorem_hash": "sha256:...",
  "compiler_version": "0.1.0",
  "token_budget": 56000,
  "token_count": 43192,
  "items": [
    {
      "source_type": "claim",
      "source_id": "clm-...",
      "view": "capsule",
      "score": 9.31,
      "reasons": ["logical_dependency", "critic_accepted"],
      "status": "critic_accepted",
      "tokens": 418
    }
  ],
  "high_scoring_exclusions": [
    {"source_id": "att-...", "reason": "redundant_under_mmr"}
  ],
  "packet_sha256": "..."
}
```

The manifest enables three forms of audit:

- **reproduction:** rebuild exactly what the worker saw;
- **diagnosis:** determine whether a bad result followed from bad reasoning or missing context;
- **evaluation:** learn which retrieval features correlate with useful outputs.

5.8 Context feedback loop

Worker output should include context_use records:

```
{
  "source_id": "clm-123",
  "use": "essential premise",
  "effect": "closed subgoal G2"
}
```

A critic can identify missing context retrospectively. The engine then stores a context_miss event, such as “sibling counterexample ATT-19 would have falsified this move before dispatch.” These events train retrieval weights and Hyperon guidance rules without requiring end-to-end learning from theorem completion alone.

6. The Autonomous Proof-Cycle Algorithm

6.1 The unit of autonomy

The main OmegaClaw coordinator should not micromanage individual metagraph records. Its normal action is one call: `proof-cycle(proof_id)`

A cycle is deliberately finite. Persistent autonomy emerges by repeatedly executing finite, auditable cycles under time and cost budgets. This avoids an unbounded agent loop inside a single tool call and gives the scheduler a chance to react after every result.

6.2 Reference algorithm

A reference cycle is:

```
def proof_cycle(proof_id: str) -> CycleDigest:
    lease = store.lease_best_frontier_move(proof_id)
    if lease is None:
        return audit_or_expand_frontier(proof_id)

    attempt = store.create_attempt_from_lease(lease)
    packet = context_compiler.compile(attempt, lease)
    persona = router.choose_persona(packet, lease.move)

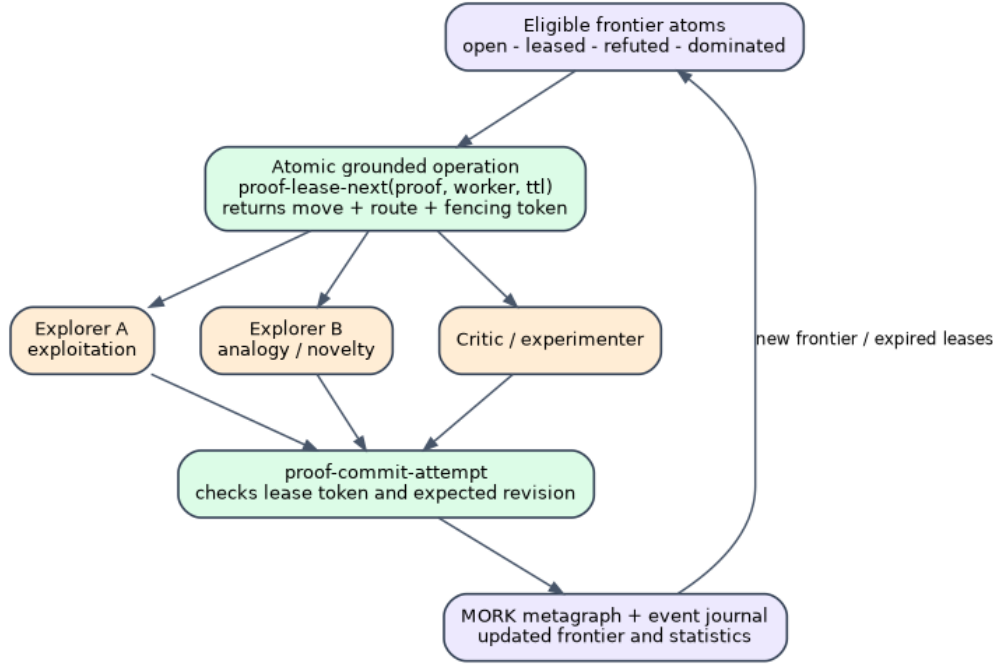
    worker_result = dispatcher.run(
        persona=persona,
        context_path=packet.path,
        result_path=attempt.result_path,
    )
    store.save_exact_dispatch_record(attempt, worker_result)

    proposal = schema_validator.parse(attempt.result_path)
    if not proposal.valid:
        return reject_and_requeue(lease, proposal.error)

    critic = maybe_run_critic(proposal, attempt)
    commit = commit_gate.apply(
        proposal=proposal,
        critic=critic,
        expected_revision=lease.base_revision,
        lease_id=lease.lease_id,
        fencing_token=lease.fencing_token,
    )

    mork_maintenance.update_indexes_frontier_routes_and_taint(commit)
    scheduler.update_statistics(commit)
    summaries.update_from_source_ids(commit)
    return commit.digest
```

The atomic lease is crucial:



Atomic frontier leasing with fencing tokens

The system must not query the best move and mark it leased in two separate transactions.

6.3 Selecting the next action

The scheduler considers only moves whose prerequisites are currently satisfiable and whose state is not tainted. A baseline priority is:

$$P(m) = w_p p_m + w_i i_m + w_n n_m + w_v v_m - w_c c_m - w_d d_m - w_f f_m.$$

where:

- (p_m): estimated promise;
- (i_m): expected information gain;
- (n_m): novelty relative to recent attempts;
- (v_m): expected verification value;
- (c_m): cost;
- (d_m): duplication with active leases;
- (f_m): repeated-failure penalty.

Promise is not the same as truth confidence. A low-probability counterexample search can have high information value. A formalization attempt may have modest probability of immediate success but high value because it exposes hidden assumptions.

6.4 Stagnation and backtracking policy

Autonomous continuation needs explicit thresholds. A default branch policy might be:

After 1 failed attempt:

classify the earliest obstruction and retry only if a concrete repair exists.

After 2 non-progress attempts:

invoke the five-mechanism procedure; queue all viable mechanisms.

After 3 failures with the same mechanism family:
 reduce the branch score and backtrack to the nearest OR ancestor.

After 5 branch failures under the same root strategy:
 request a root-level portfolio using different technique families.

After global stagnation:
 run an audit, Hyperon creative intervention, literature refresh,
 and theorem-statement/assumption review before spending more tokens.

Thresholds should be measured in outcomes, not merely model turns. Rephrasing the same argument is one mechanism family, even if several workers attempted it.

6.5 The five-mechanism procedure

The “suppose a supermind completed the proof” idea becomes a structured branching operator:

Assume a correct proof or a decisive obstruction is reachable from the current state. Generate five genuinely different mechanisms.

For each mechanism provide:

- cognitive operator and technique family;
- central idea;
- exact bridge lemma or reduction;
- why that lemma would close the current gap;
- prerequisite claim IDs;
- earliest inexpensive falsification test;
- predicted information gain if false;
- promise, cost, and mathematical risk;
- relation to existing branches;
- whether it proves, bypasses, generalizes, or reframes the target.

Diversity is enforced by technique tags and graph distance. Five cosmetic variations of induction do not satisfy the procedure. Hyperon can seed the portfolio with symbolic analogies and abductive candidates; an LLM makes them mathematically legible.

6.6 Commit gate

The commit gate checks:

- event and worker-result schemas;
- stable IDs and proof namespace;
- expected proof revision;
- active lease ID and fencing token;
- literal theorem identity and assumption changes;
- legal claim-status transitions;
- existence and hashes of referenced artifacts;
- dependency references and prohibited cycles;
- distinction between empirical evidence and deduction;
- distinction between literal proof, reduction, bypass, stronger substitute, barrier, and failure;
- taint and stale-formalization consequences;
- completion candidates for unresolved dependencies and open states.

The gate then emits one atom-operation event. Workers never issue raw `add-atom` or `remove-atom` calls against the committed layer. Hyperon may write through a narrow speculative-hypothesis API, but even those writes are journaled and namespace checked.

6.7 Critic policy

Not every exploratory thought needs a critic, but status-changing results do. A practical policy is:

- critic required for every proposed supported claim used by another branch;
- critic required before marking an AND subgoal closed;
- two independent critics for a proof-complete candidate;
- targeted critic after numerical or symbolic evidence suggests a theorem;
- automatic adversarial review when a worker changes assumptions or quantifiers;
- formal verifier supersedes local stylistic criticism only for the exact formal statement.

The critic should receive a context packet that omits the explorer’s rhetorical confidence and highlights dependencies, theorem kernel, and possible failure classes.

6.8 Milestone communication

The user should not receive a message after every cycle. OmegaClaw reports when one of the following occurs:

a durable claim is promoted
 a major branch is opened, refuted, bypassed, or superseded
 a conceptual compression replaces many local calculations
 a new obstruction changes the search program
 a resource or semantic ambiguity requires a human decision
 a proof-complete candidate is ready for audit
 configured time, cost, or safety limits are reached

A milestone report includes IDs and links to artifacts, but is written for humans. The rest remains in the workspace.

7. Prompt and Skill Architecture

7.1 Two layers: behavioral specifications and executable skills

The portable behavioral layer is maintained as version-controlled Markdown `SKILL.md` files. The executable OmegaClaw layer follows OmegaClaw’s current custom-skill pattern: a catalogue description, a MeTTa definition, and optional Python or Prolog glue through `py-call`.¹²

The skill set includes a dedicated `proof-mork-store` specification and a `proof-hyperon-creative` skill. The Markdown files are normative behavioral contracts; the executable wrappers are deliberately thin and call the proof-store service.

7.2 Minimal OmegaClaw skill surface

The coordinator should see only a few high-level skills:

```
(= (proof-create $spec $workspace)
  (py-call (mathproof.omegaclaw_bridge.proof_create $spec $workspace)))

(= (proof-cycle $proof $workspace)
  (py-call (mathproof.omegaclaw_bridge.proof_cycle $proof $workspace)))

(= (proof-status $proof $workspace)
  (py-call (mathproof.omegaclaw_bridge.proof_status $proof $workspace)))

(= (proof-audit $proof $workspace)
  (py-call (mathproof.omegaclaw_bridge.proof_audit $proof $workspace)))
```

¹² ASI Alliance, *Writing a Custom Skill*, OmegaClaw tutorial, accessed 15 July 2026.
<https://raw.githubusercontent.com/asi-alliance/OmegaClaw-Core/main/docs/tutorial-03-writing-a-custom-skill.md>


```
(= (proof-export $proof $destination $workspace)
  (py-call (mathproof.omegaclaw_bridge.proof_export
            $proof $destination $workspace)))
```

Lower-level lease and commit operations are useful for debugging and specialized orchestration, but normal autonomy should call `proof-cycle`. Paths and identifiers must be quoted and validated inside the grounded implementation.

7.3 Coordinator prompt

The persistent coordinator prompt should emphasize policy, not mathematical detail:

You are the coordinator of a long-horizon mathematical proof program.
The authoritative state is the local proof workspace, not chat history.

At every cycle:

1. Read proof status and honor the literal theorem kernel.
2. Invoke `proof-cycle` unless an audit, semantic clarification, or resource decision is required.
3. If one move is clearly best, pursue it.
4. If several moves are viable, pursue one and preserve the others.
5. After repeated non-progress, invoke structured creative expansion and backtracking rather than paraphrasing the same idea.
6. Distinguish established, provisional, empirical, critic-accepted, formally verified, refuted, and retracted claims.
7. Never silently change assumptions, quantifiers, constructivity, or the literal target.
8. Treat bypasses and stronger route-level substitutes as progress, not as completion of the old target without an implication chain.
9. Ask Hyperon for creative guidance when conceptual compression, analogy, obstruction inversion, or hybridization is likely useful.
10. Communicate only durable milestones, ambiguities, contradictions, resource limits, and completion candidates.

7.4 Worker roles

The first deployment benefits from a small set of sharply distinguished personas.

Persona	Primary objective	Typical tools
Explorer	advance one selected move	local record lookup, Python request
Critic	find the earliest exact gap	claim/dependency lookup
Counterexample hunter	falsify a claim cheaply	Python, SAT/SMT, small search
Analogy explorer	transfer a mechanism from another branch/domain	Hyperon suggestions, literature notes
Experimenter	execute a preregistered computation	Python sandbox
Formalizer	translate one claim to Lean	Lean toolchain
Adjudicator	compare conflicting reports	read-only proof records

The worker prompt includes a non-negotiable output schema. Workers should not be asked to update the committed metagraph, rename files, or decide authoritative status.

7.5 proof-step output contract

A worker result is bound to the exact lease and proof revision:

```
{
  "proof_id": "p17",
  "attempt_id": "a411",
  "move_id": "m204",
  "state_id": "s101",
  "route_id": "r52",
```

```

"lease_id": "lease-771",
"fencing_token": 14,
"base_revision": 982,
"summary": "...",
"disposition": "reduction",
"result_relation": "reduces-target",
"new_claims": [],
"new_states": [],
"new_moves": [],
"closed_goals": [],
"remaining_gaps": [],
"obstructions": [],
"queued_alternatives": [],
"artifacts": [],
"assumption_changes": [],
"suggested_critic_focus": [],
"confidence_rationale": "..."
}

```

Workers may initially propose only conservative claim statuses. The complete machine-readable schema is included in the companion bundle.

7.6 Critic prompt

The critic is not asked whether the proof “looks good.” It performs a structured audit:

1. Restate the exact claim and assumptions.
2. List every dependency actually used.
3. Locate the earliest unsupported inference.
4. Test quantifier order, edge cases, sign/direction changes, and hidden regularity or finiteness assumptions.
5. Distinguish a repairable exposition gap from a mathematical gap.
6. Check whether the result proves, bypasses, or merely motivates the target.
7. Propose the smallest decisive repair or falsification test.
8. Return a verdict with cited record IDs.

A critic may recommend critic-accepted only for the exact statement reviewed; the commit gate performs the actual status transition.

7.7 proof-supermind and proof-hyperon-creative

The proof-supermind skill serves as an LLM-centered strategy expander. The proof-hyperon-creative skill precedes or accompanies it when symbolic guidance is enabled. Its responsibilities are:

- select the current obstruction and concept neighborhood directly from the MORK metagraph;
- invoke a bounded portfolio of creative operators;
- return diverse suggestion packets with evidence and uncertainty;
- request LLM elaboration only for suggestions that pass novelty and coherence filters;
- record which suggestions were adopted and their outcomes.

This pairing gives a useful division of labor: Hyperon searches structural relations and composes heuristic mechanisms; the LLM supplies flexible mathematical language and local derivations.

7.8 Prompt versioning and tests

Every prompt and skill has a semantic version and content hash. Attempts record both. Regression fixtures should include:

- a branch with a hidden quantifier error;
- a route-level bypass that must not close the literal target;
- a computational pattern that requires a proof rather than extrapolation;
- several queued alternatives where only one is pursued;

- a retracted claim with dependent branches;
- a Hyperon suggestion that is novel but unsupported;
- a ThreadKeeper digest that points to a full result file.

Prompt changes can then be evaluated on replayed proof states instead of judged only by prose quality.

8. Hyperon as a Creative Mathematical Partner

8.1 Hyperon's role in the MORK-native architecture

The search, justification, provenance, concept, and speculative structures inhabit a MORK-backed PeTTa space. Hyperon can therefore reason over the same stable IDs and relations used by the scheduler and context compiler. No periodic semantic translation is required, so branch distinctions and provenance remain explicit throughout the reasoning process.

Hyperon has three roles:

1. exact and uncertain relevance for context and frontier ranking;
2. diagnosis of repeated failures, barriers, and missing dependencies;
3. generation of genuinely new mathematical hypotheses and moves.

The third role is intended to absorb part of the human research-director function.

8.2 Speculative hypothesis layer

Creative results are first-class objects, not unstructured paragraphs:

```
(proof p17 layer speculative hypothesis h701)
(proof p17 hypothesis-kind h701 analogy-transfer)
(proof p17 targets-subgoal h701 s103)
(proof p17 source-concept h701 rank-collapse)
(proof p17 source-concept h701 spectral-bias)
(proof p17 suggests-mechanism h701 mech27)
(proof p17 prerequisite h701 c211)
(proof p17 expected-consequence h701 c_candidate)
(proof p17 falsification-test h701 test44)
(proof p17 abductive-strength h701 score074)
(proof p17 novelty h701 score081)
```

This representation supports provenance, deduplication, review, expiration, later outcome analysis, and learning by mechanism type.

8.3 Creative operators

Missing-lemma abduction

Given an active dependency gap, hypothesize the weakest bridge lemma and one or more stronger, structurally cleaner variants. Each proposal must state why it would close the gap and what could falsify it cheaply.

Analogy transfer

Find a subgraph in another branch or proof with similar relational roles, for example:

```
entropy barrier -> deterministic shield -> rare residual set -> random amplification
```

Map source roles to target concepts and record where the analogy may break.

Concept blending and hybridization

Combine mechanisms that address different resources. The supplied proof effort’s “random core plus deterministic shield” pattern is a canonical example: one component handles broad entropy while another eliminates a huge structured adversary class.

Representation change and dualization

Propose a new algebra, basis, dual object, geometric model, probabilistic view, category, ideal, flow, energy, or game in which the obstruction becomes simpler. The output includes an explicit translation task.

Obstruction inversion

Turn repeated failure into a theorem or invariant. A saturation point in a union-bound method may become a barrier theorem that precisely states what new structure a successful route must exploit.

Counterexample and extremal construction

Generate a minimal finite model or extremal family that distinguishes nearby statements. Connect the proposal to a bounded experiment when enumeration is useful.

Strengthen to simplify; generalize then specialize

A stronger theorem may expose a clean invariant. Hyperon may propose it, but the graph must retain the implication back to the literal target. Likewise a generalization is useful only with a recorded specialization path.

Local-to-global and structure-change operators

Other useful operators include compactness, induction on complexity, local certificates, symmetry breaking, quotienting, random restriction, and decomposition by resource or obstruction type.

8.4 PLN support and current limits

PLN can attach strength/confidence and evidence to uncertain support, analogy, and abductive relations. OmegaClaw’s current PLN documentation describes supported inference shapes and also notes limitations around more general backward or abductive inference.¹³ The architecture does not assume every creative operator already exists as a built-in rule. Domain-specific MeTTa/MM2 rules or a grounded creative service may emit the same provenance-preserving hypothesis schema.

The key separation is:

```

PLN truth value      -> uncertain heuristic support
claim protocol status -> accepted mathematical state

```

No numeric threshold promotes a claim directly.

8.5 Deliberation pipeline

```

active MORK neighborhood
  -> bounded PLN and creative rules
  -> hypothesis generation
  -> novelty/redundancy filtering
  -> LLM mathematical elaboration
  -> critic and cheap falsification
  -> speculative move queue
  -> selected proof attempt
  -> outcome returned to MORK/PLN

```

¹³ TrueAGI, *Probabilistic Logic Networks: MeTTa-native PLN for Hyperon*, GitHub repository, accessed 15 July 2026. <https://github.com/trueagi-io/PLN>

An LLM is especially useful in the middle: translating a symbolic analogy or missing-lemma relation into definitions, exact statements, and a first proof plan. The LLM does not erase the hypothesis’s symbolic provenance.

8.6 Five-mechanism procedure with Hyperon

At a difficult state, the coordinator asks Hyperon and an LLM for five genuinely distinct mechanisms. For each:

- core idea;
- exact missing lemma or reduction;
- source concepts and analogous branches;
- why it would close or clarify the current gap;
- prerequisite claim IDs;
- first inexpensive falsification test;
- expected information gain if false;
- promise, cost, novelty, and risk;
- relation to existing moves;
- first executable step.

All viable options enter the speculative queue. One is selected immediately; the others remain addressable for backtracking.

8.7 Human guidance as learnable search data

When a human says “try the next sharp move,” proposes an analogy, rejects a representation, or points out a hidden obstruction, store:

- state and route;
- cognitive operation implied;
- concepts or branches highlighted;
- chosen move;
- later critic, experiment, and proof outcomes.

Hyperon can induce conditional guidance patterns from these episodes. This is how the system may learn when many low-order calculations call for conceptual compression, when a barrier should be inverted, or when a random/deterministic hybrid is promising.

8.8 Creativity controls

Control creative search by:

- maximum hypotheses per run;
- novelty floor and redundancy penalty;
- theorem-kernel constraint checking;
- resource and falsification budgets;
- mandatory source provenance;
- hypothesis expiry and review;
- separation of speculative and committed layers;
- human vetoes over forbidden reformulations.

The objective is not maximal weirdness. It is novel, relevant, testable, mathematically consequential search.

8.9 Outcome learning

Record whether each hypothesis:

- was elaborated into a coherent move;
- survived criticism;
- was falsified cheaply;
- closed or simplified a subgoal;
- produced a reusable claim;
- contributed to a Lean-verified theorem;
- merely duplicated an existing route.

These outcomes can train better rule weights, retrieval, mechanism selection, and scheduling without rewriting proof history.

8.10 Example: N105–N109 as creative mechanism evolution

The supplied proof effort exhibits precisely the transitions this layer should learn. Many fixed-moment calculations were compressed into a general Cayley–Bacharach mechanism. A literal random-map statement was bypassed by a deterministic algebraic shield. A spectral shield and random quadratic core then produced a stronger entropy mechanism. A Hall-expander shield increased algebraic immunity and exposed a cleaner high-degree residual target. Hyperon should represent these not only as claims, but as reusable mechanism transformations: conceptual compression, obstruction inversion, resource decomposition, and shield hybridization.

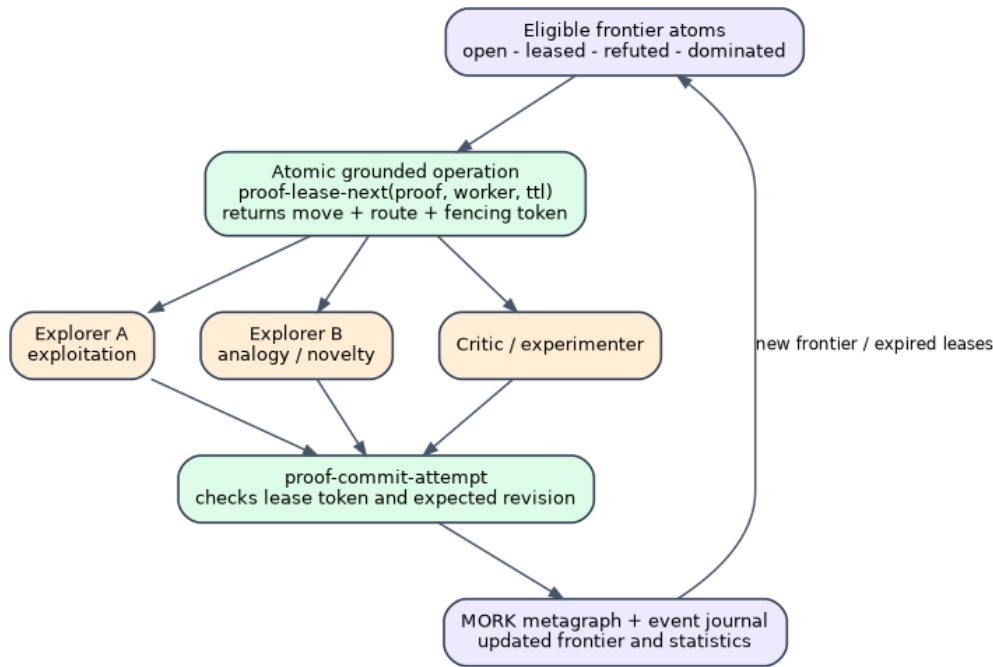
9. Parallel Multi-Agent Proof Search

9.1 Why the chess-engine analogy is useful but incomplete

The proposed system resembles a chess engine in that many candidate continuations compete for bounded search resources, earlier positions are revisited, and parallel workers share evaluations. The analogy breaks in important ways:

- theorem-proving moves are generated dynamically rather than drawn from a fixed legal-move set;
- a move may create several mandatory subgoals, so the graph is AND/OR rather than alternating minimax;
- the value of a partial proof is difficult to estimate and may be dominated by a newly discovered lemma;
- failed branches can be valuable because they reveal obstructions;
- different branches can merge through shared claims;
- proof checking is asymmetric: a complete formal certificate is decisive, while heuristic value estimates are not.

The right initial model is therefore **parallel best-first search over an AND/OR DAG with transpositions**, not direct importation of a chess search algorithm.



Parallel leasing

Parallel workers share a leased frontier and authoritative claim ledger.

9.2 Frontier representation

The frontier is a MORK-derived candidate space, not merely a flat list. Committed atoms identify open, reopened, leased, refuted, dominated, and exhausted moves. MORK union, subtraction, restriction, and transformation operations construct an exact eligible set. Python or PLN then assigns or updates numeric scheduling values.

A frontier entry records move ID, state ID, one or more route IDs, technique family, prerequisite status, estimated promise, novelty, information gain, cost, verification value, repeated-failure count, and current lease state. The score snapshot used for a lease is retained for later scheduler evaluation.

9.3 Leases and concurrency

The only safe selection primitive is atomic:

```
proof-lease-next(proof_id, worker_id, ttl)
```

Under one proof lock, the service expires stale leases, constructs the eligible frontier, selects a move and route, increments a fencing token, marks the move leased, appends the event, applies the atoms to MORK, and returns the lease.

A worker result includes lease ID, fencing token, and base revision. A late result from token 13 is rejected after token 14 has been issued. The attempt and artifacts are preserved and may be reconsidered under a new move, but they cannot complete the obsolete lease.

For two or three workers, one proof-store process and one MORK space are sufficient. Most wall time is spent in model and tool calls, so the short critical section is inexpensive.

9.4 Cognitive diversity

Parallelism is most useful when workers have different search biases. The scheduler can reserve capacity for modes such as:

exploitation: highest current promise
 exploration: high novelty or uncertainty
 analogy: cross-branch and external transfer
 adversarial: counterexample or gap hunting
 empirical: inexpensive computational discrimination
 formal: Lean translation or exact symbolic verification

With three workers, a useful default is one exploitation worker, one exploration/analogy worker, and one critic or experiment worker. A role-diversity penalty prevents all workers from becoming near-identical generalist explorers.

9.5 Duplicate detection and transpositions

Duplicate detection uses both semantic and structural signals.

- Stable state signatures identify exact transpositions.
- MORK route projections show several paths reaching the same state.
- Claim and assumption sets help distinguish superficially similar but logically different states.
- Embeddings detect paraphrased move descriptions.
- Hyperon concept mappings detect analogical rather than identical duplication.

When two branches converge, retain both route histories and share the state and claims. Do not merge states merely because their prose is similar.

9.6 AND/OR propagation

The scheduler should propagate progress through the graph.

- An OR node is solved when any child route closes it.
- An AND node is solved only when every required child is closed.
- A refuted child of an OR node lowers that route but leaves alternatives.
- A refuted required child of an AND decomposition invalidates that decomposition and reopens the parent strategy.
- A bypass closes a route-level objective only if that objective's policy accepts the bypass relation; it does not close the literal target automatically.

A simple proof-number-inspired estimate can be maintained:

OR proof estimate = min(child proof estimates)
 AND proof estimate = sum(child proof estimates)
 OR disproof estimate = sum(child disproof estimates)
 AND disproof estimate = min(child disproof estimates)

These estimates are heuristic when subgoals are generated dynamically, but they are useful for prioritizing nearly closed decompositions.

9.7 From best-first search to PUCT

Best-first search is recommended initially because every decision can be explained. After the system has enough attempt data, the priority can include a PUCT-like exploration term:

$$U(s, m) = Q(s, m) + c_{puct} P(s, m) \frac{\sqrt{N(s)}}{1 + N(s, m)}.$$

Here $Q(s, m)$ is empirical outcome value, $P(s, m)$ is a prior from the LLM/Hyperon scheduler, $N(s)$ counts visits to the state, and $N(s, m)$ counts attempts of the move. For proof search, q should combine multiple outcomes rather than binary win/loss:

```
verified claim production
subgoals closed
branch pruning
counterexamples found
reusable lemmas
context and verification cost
```

Progressive widening controls the number of newly generated moves. The scheduler expands more alternatives only as a state receives more visits or when the existing children stagnate.

9.8 Shared learning without shared confusion

Workers share the committed metagraph, not one another's unfiltered conversations. New accepted claims, refutations, barriers, and artifacts become available through MORK and context compilation. Speculative hypotheses remain visibly speculative. This permits rapid cross-branch learning without allowing one model's unsupported claim to become another model's premise.

A global concept/mechanism space can share verified lemmas and search experience across theorems. Imports into a proof namespace are explicit, versioned, and provenance bearing.

9.9 Scaling beyond a few workers

Beyond a few workers, shard by theorem: one proof-store owner and MORK space per theorem, plus a separate global knowledge service. Add asynchronous job IDs, polling, cancellation, and heartbeat renewal to the dispatch layer. Keep the same attempt-result and lease contracts.

At larger scale, contention is not the only concern. Search quality can degrade through correlated move generation. Use model/persona diversity, technique quotas, novelty constraints, and progressive widening. Scale only when objective proof progress per unit cost improves.

10. Reproducible Computation and Artifact Management

10.1 Computation as a first-class proof action

In exploratory mathematics, a “proof step” may be a Python program that enumerates small cases, estimates a distribution, searches for a counterexample, verifies a combinatorial identity, or discovers a pattern. These actions should not be pasted into the transcript and forgotten. They are attempts with hypotheses, methods, outputs, and interpretation limits.

The artifact system is designed so that another machine can reproduce the run and another agent can understand what conclusion it licenses.

10.2 Experiment specification

Every experiment begins with `experiment-spec.json`:

```
{
  "experiment_id": "exp-...",
  "attempt_id": "att-...",
  "question": "Does the proposed core-dimension bound fail for n <= 8?",
  "hypotheses": [
    {"id": "H1", "text": "No counterexample exists in the exhaustive cell."},
    {"id": "H2", "text": "A counterexample exists and identifies a missing hypothesis."}
  ]
}
```

```

],
"method": "exhaustive enumeration",
"parameter_space": {"n": [1, 8]},
"randomness": {"used": false},
"success_criteria": "Complete enumeration with independent count checks.",
"interpretation_limits": [
  "Passing finite cells does not prove the asymptotic theorem."
],
"requested_outputs": ["results.json", "counterexamples.json"]
}

```

The specification is a lightweight preregistration. It prevents a worker from running an ambiguous computation and retrofitting a conclusion to whatever output appears.

10.3 Attempt artifact layout

```

attempts/<attempt-id>/artifacts/
  experiment-spec.json
  code/
    main.py
    helpers.py
    tests.py
  input/
  command.txt
  environment.json
  random-seeds.json
  stdout.txt
  stderr.txt
  results.json
  tables/
  figures/
  interpretation.md
  sha256sums.txt

```

`environment.json` records Python version, operating system, package lock hash, relevant CPU/GPU information, and tool versions. Container image digests or Nix/Guix manifests can be added later.

10.4 Sandboxed execution

Experiment workers should run with:

```

no network by default
read-only access to the compiled context and declared input artifacts
write access only to the attempt directory
CPU, memory, wall-clock, and output-size limits
an allowlist of executables and Python packages
captured stdout and stderr
exit status and signal recording

```

Network access for literature or package installation is a separate, governed action. This separation protects the local proof repository and improves reproducibility.

10.5 Numerical evidence semantics

The result contract classifies computational evidence:

Evidence type	Permitted conclusion
random sampling	empirical support or counterexample found
exhaustive finite search	theorem for the enumerated finite domain
exact symbolic computation	equality or certificate under the tool's stated semantics
interval arithmetic	bounded numerical claim with recorded precision
independently checkable certificate	claim conditional on certificate checker correctness
heuristic optimizer	candidate or upper/lower bound, not proof without certification

Evidence type	Permitted conclusion
A successful random experiment never promotes an asymptotic claim to <code>critic_accepted</code>. It may increase scheduler promise or motivate a lemma.	
10.6 Content-addressed storage Large or repeated artifacts are stored by hash: <code>objects/sha256/ab/cd/<full-hash></code> Artifact records point to the hash and include a logical filename. This deduplicates repeated datasets and ensures that a claim references immutable bytes. Ordinary Git tracks manifests and small source files; large objects can use Git LFS, an object store, or periodic archive snapshots.	
10.7 Deterministic exports and backup A proof export contains: - normalized theorem specification - current graph and claim ledger as sorted JSON - human-readable status report - all prompt and skill versions used - small artifacts and manifests - content hashes for large artifacts - atom, event, and manifest schema versions - Hyperon ontology and creative-rule versions - Lean toolchain lock files The live journal can be copied through a transactionally safe checkpoint or backup procedure, but Git should receive a deterministic export rather than a partially written tail. Secrets, provider tokens, and personal connector data are excluded by policy.	
10.8 Literature and external source artifacts Literature notes are stored as artifacts with bibliographic metadata, access date, exact passages or theorem numbers, and a short relevance statement. They are not silently merged into the claim ledger. Imported claims identify whether they are assumed as background, independently checked, or merely suggested by a source.	
11. Lean 4 Integration	
11.1 Incremental formalization, not a final monolith Lean should enter the workflow at claim granularity. Waiting until a hundred-page informal proof is “finished” and then attempting one massive translation is expensive and obscures where formalization exposes a problem. Stable lemmas should be formalized as the proof grows. Lean 4 is a theorem prover and programming language with a small trusted kernel, and <code>mathlib</code> provides a large community library and standard project tooling.¹⁴¹⁵ The proposed agent uses Lean as an independent checker for exact formal declarations, not as a blanket endorsement of nearby prose.	

¹⁴ Lean project, *Lean 4 theorem prover and programming language*, official documentation, accessed 15 July 2026. <https://lean-lang.org/>

¹⁵ Mathlib community, *mathlib4*, GitHub repository and documentation, accessed 15 July 2026. <https://github.com/leanprover-community/mathlib4>

11.2 Claim-to-Lean mapping

A claim record gains:

```
lean_name
lean_statement_path
dependency declaration names
namespace and imports
toolchain hash
mathlib revision
formalization status
last compiler output
```

Stable naming matters. A claim ID can map to a generated declaration name such as:

```
namespace Proof_7F31
```

```
theorem clm_01J9A_core_dimension_bound
  (S : Finset (Fin n -> ZMod 2))
  (hcb : QuadraticCB S) :
  affineSpanDimension S <= ... := by
  ...
```

```
end Proof_7F31
```

The informal and formal statements are displayed side by side during review. A human or critic must confirm that the translation preserves the intended quantifiers and definitions.

11.3 Formalization loop

```
select stable claim
-> generate a translation draft
-> run theorem-statement equivalence review
-> invoke Lean on a small file
-> classify errors
-> repair translation or expose mathematical gap
-> commit source and logs
-> promote exact claim to lean_verified when the kernel accepts it
```

The formalizer receives only dependencies already formalized or explicitly admitted in a separate sandbox mode. Production verification must reject sorry and untracked axioms.

11.4 Error classification

Lean failures are classified as:

```
translation ambiguity
missing definition or library lemma
elaboration/type mismatch
tactic search failure
resource timeout
inconsistent assumptions
likely mathematical gap
formal statement stronger than informal claim
```

This classification determines the next move. A missing import should not lower mathematical promise. A discovered counterexample to the formal statement should reopen the claim.

11.5 Tool interface

A dedicated tool is preferable to a generic shell command:

```
{
  "tool": "lean-check",
  "proof_id": "proof-...",
  "claim_id": "clm-...",
```

```

"source_path": "lean/Clm01J9A.lean",
"toolchain": "leanprover/lean4:...",
"mathlib_revision": "...",
"timeout_seconds": 120,
"deny_sorry": true
}

```

The result includes diagnostics, imported declarations, axioms used, timing, and a source hash. Lake project files and lean-toolchain are pinned.¹⁶

11.6 Formalization strategy

The scheduler should prefer claims that are:

- central dependencies used by many branches;
- stable enough that their statement is unlikely to change;
- locally self-contained;
- susceptible to hidden edge cases;
- already close to mathlib abstractions;
- bottlenecks whose verification would substantially raise trust.

Formalizing every exploratory lemma immediately is wasteful. Formalizing only the final theorem is too late. The claim graph supplies the middle ground.

11.7 Interaction with Hyperon and LLMs

Lean diagnostics become typed obstructions in the proof graph and Hyperon space. Repeated formalization errors can trigger suggestions such as:

```

strengthen an invariant;
split a theorem into an algebraic and a finite-combinatorial lemma;
replace an extensional equality by an isomorphism;
search mathlib for a canonical abstraction;
weaken a statement to match the actual induction hypothesis.

```

LLMs remain useful for translation and tactic synthesis, but every accepted result is determined by Lean's checker.

12. Practical OmegaClaw, PeTTa/MORK, and ThreadKeeper Deployment

12.1 Recommended repository layout

```

math-proof-agent/
  pyproject.toml
  src/mathproof/
    store.py
    event_log.py
    mork_adapter.py
    scheduler.py
    context_compiler.py
    commit_gate.py
    artifact_store.py
    hyperon_adapter.py
    threadkeeper_adapter.py
    lean_adapter.py
  omegaclaw/
    proof_skills.metta
  mork/
    proof_atoms.metta
    indexes.mm2

```

¹⁶ Lean project, *Lake build system and package manager*, official documentation, accessed 15 July 2026. <https://lean-lang.org/doc/reference/latest/Build-Tools-and-Distribution/Lake/>

```

dependency_reachability.mm2
taint_propagation.mm2
prompts/
skills/
  proof-orchestrator/SKILL.md
  proof-mork-store/SKILL.md
  proof-context/SKILL.md
  proof-step/SKILL.md
  proof-critic/SKILL.md
  proof-supermind/SKILL.md
  proof-hyperon-creative/SKILL.md
  proof-experiment/SKILL.md
  lean4-formalizer/SKILL.md
  proof-audit/SKILL.md
threadkeeper/personas-subagent/
schemas/
config/
tests/

```

The proof workspace itself is separate from code so it can be backed up, mounted with stricter permissions, and migrated independently.

12.2 Building PeTTa with MORK support

PeTTa documents MORK as an optional backend enabled through its build process, and a MORK space can be manipulated using atomspace operations and `mm2-exec`.¹⁷ Pin PeTTa, MORK, and `mork_ffi` revisions in the proof-store deployment. Record those versions in every checkpoint.

Do not let several independent workers write directly to the global `&mork` handle. The proof-store process owns MORK access and exposes transactional high-level operations.

12.3 OmegaClaw integration

OmegaClaw custom skills currently combine a skill-catalogue description, a MeTTa definition, and optional Python or Prolog implementation.¹⁸ The integration file exposes thin wrappers around the proof-store API. The coordinator prompt remains stable even if internal storage or worker dispatch evolves.

A typical OmegaClaw loop result is compact:

```

proof-cycle: p17 a411 committed; c308 provisional; s102 and s103 open;
Hyperon h701 queued; next critic due; full report in attempts/a411/

```

Ordinary OmegaClaw memory may store user preferences and portfolio-level history. It does not replace the theorem workspace.

12.4 ThreadKeeper integration

The parent creates an attempt directory and dispatch contract:

```

context path
context-manifest path
result path
artifact directory
lease ID, fencing token, and base revision

```

The bounded persona writes full results to disk and returns a single-line digest. Workers cannot mutate MORK, parent memory, or delegate recursively. A production launcher additionally enforces path, network, CPU, memory, and wall-clock limits.

¹⁷ TrueAGI, *PeTTa* repository and *MORK spaces* documentation, accessed 15 July 2026. The documented MORK backend supports an `&mork` space, ordinary atom operations, matching, and `mm2-exec`. <https://github.com/trueagi-io/PeTTa> and <https://github.com/trueagi-io/PeTTa/wiki/MORK-spaces>

¹⁸ ASI Alliance, *Writing a Custom Skill*, OmegaClaw tutorial, accessed 15 July 2026. <https://raw.githubusercontent.com/asi-alliance/OmegaClaw-Core/main/docs/tutorial-03-writing-a-custom-skill.md>

12.5 Proof-store service and transport

For an initial local deployment, use one Python process per active proof and a Unix-domain socket or local HTTP API. The service owns:

- proof lock and revision;
- MORK adapter;
- journal and checkpoints;
- leases and fencing tokens;
- context compilation;
- commit and audit;
- artifact manifests.

A stronger future MORK service may supply named spaces, snapshots, compare-and-swap, and atomic batches directly. The surrounding contracts remain useful.

12.6 Configuration

```
store:
  semantic_backend: mork
  durability_backend: jsonl
  one_process_per_proof: true
  event_hash_chain: true
  maintenance_iteration_limit: 64

scheduler:
  policy: best_first_and_or
  workers: 3
  lease_ttl_seconds: 1800
  virtual_loss: 0.15

context:
  token_budget: 60000
  structural_weight: 4.0
  pln_weight: 1.3
  semantic_weight: 2.5
  include_divergent_strategy_when_stale: true

hyperon:
  enabled: true
  write_layer: speculative
  max_hypotheses_per_run: 12

lean4:
  enabled: false
  command: lake env lean
```

12.7 Operational commands

```
mathproof create theorem-spec.yaml --workspace WORKSPACE
mathproof status --proof-id P --workspace WORKSPACE
mathproof lease --proof-id P --worker W --workspace WORKSPACE
mathproof prepare-attempt --proof-id P --worker W --workspace WORKSPACE
mathproof commit-result result.json --proof-id P --workspace WORKSPACE
mathproof replay --proof-id P --workspace WORKSPACE
mathproof audit --proof-id P --workspace WORKSPACE
```

Additional production commands should expose frontier inspection, hypothesis review, context replay, artifact verification, checkpointing, and deterministic export.

12.8 Observability

Monitor:

- proof revision and journal tail hash;
- MORK committed and generated-view digests;
- frontier by mechanism and status;
- active, expired, and conflicting leases;
- attempts and objective outcomes by persona/model;
- context source-channel usage;
- Hyperon hypothesis adoption and falsification;
- claim statuses and taint cones;
- artifact verification failures;
- Lean queue and stale formalizations;
- human interventions and cost.

Every dashboard item links to stable IDs.

12.9 Schema evolution, import, and backup

Version event schemas, atom schemas, MM2 maintenance rules, prompts, skills, and artifact manifests. A schema or workspace conversion emits explicit events or a revision-zero import stream and verifies replay. Commit journals, checkpoints, manifests, prompts, skills, and small artifacts to Git. Use Git LFS or another object store for large content-addressed objects.

13. Worked Case Study from the Supplied Proof Effort

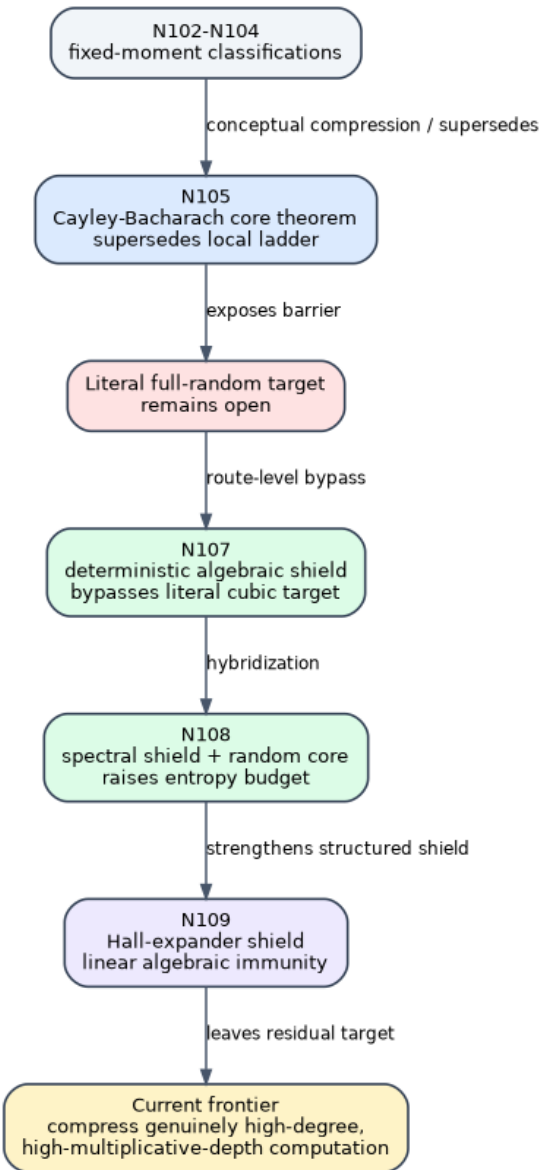
13.1 Purpose and limits of the case study

The supplied 47-page proof interaction is not used here to certify the underlying mathematical claims. It is used as a realistic example of long-horizon proof dynamics: conceptual compression, route substitution, computation, new barriers, and an unfinished frontier. The document's later pages record milestones N105 through N109 together with PDFs, LaTeX sources, Python verifiers, and logs.

13.2 Importing the transcript

Import is a conservative extraction process, not a summary pass.

1. Freeze the PDF and compute its artifact hash.
2. Create milestone attempts for N105, N107, N108, and N109.
3. Extract claims, exact open targets, newly exposed barriers, and next moves.
4. Classify relations such as supersedes, bypasses, strengthens-route, and leaves-open.
5. Register each PDF, LaTeX source, Python verifier, and verification log as a separate artifact.
6. Emit revision-zero import events and replay them into MORK.
7. Run a theorem-identity and completion audit.



MORK representation of the N105–N109 progression

The final visible continuation remains an active candidate mechanism, not a completed theorem.

13.3 N105: conceptual compression and supersession

The N105 milestone says that a sequence of fixed-moment classifications can be replaced by a general quadratic Cayley-Bacharach core theorem and an all-even-moment argument. In graph terms:

Claims N102-N104: local classifications
 Claim N105-CB: general core-dimension theorem
 Claim N105-MOM: all-even-moment theorem

N105-CB generalizes and helps derive N105-MOM
 N105-MOM supersedes route dependence on N102-N104
 N102-N104 remain historical support, not deleted

This is a textbook example for Hyperon’s conceptual-compression operator. Repeated local calculations share a structural dependency; the right new move is not “compute the next moment,” but “identify the invariant explaining all moments.”

13.4 N107: a bypass that must not close the literal target

The N107 milestone explicitly says that the literal probabilistic full-random statement remains open, while a deterministic algebraic shield achieves the security objective for a modified ensemble. The proof graph should represent:

```
Goal G-random-cubic: remains open
Move M-shield: active alternative
Claim C-shield-injective: supported/verified by attached checks
Relation M-shield bypasses G-random-cubic for route objective R
```

The route objective may be advanced or even solved while the old goal remains open. This distinction is exactly why bypasses is an explicit relation and the theorem kernel lists nearby statements that do not suffice.

13.5 N108: obstruction inversion and resource hybridization

N108 introduces a spectral shield and a random quadratic core, increasing the arbitrary-test entropy budget from a quadratic to a cubic scale. The structure is:

```
obstruction: raw moment/union-bound budget is too small
spectral shield: dense language -> sparse set of bad translations
random core: many independent translations
composition: amplify rarity of bad translations
```

This is an obstruction-inversion pattern: instead of directly counting a huge adversary class, transform each adversary into a rare bad-translation set and use a random component to avoid it. Hyperon should store the pattern independently of the particular mathematical vocabulary.

13.6 N109: a second shield and a cleaner residual target

N109 adds a sparse Hall-expander shield that raises algebraic immunity to a linear scale at controlled seed cost. It composes with the N108 mechanism. The remaining target is no longer “handle every low-degree relation,” but specifically compress high-degree, high-multiplicative-depth computation.

The correct state transition is:

```
large structured adversary subclass eliminated deterministically
cubic-scale arbitrary-test capacity retained probabilistically
residual target narrowed to high-degree/high-depth computation
literal ultimate theorem remains open
```

This is progress through **problem refactoring**. The new theorem changes what the next theorem should be.

13.7 Artifact indexing

Each milestone lists a PDF, LaTeX source, verifier, and verification log. These should be attached to the corresponding attempt and claims:

```
N108_spectral_shield_amplification.pdf
-> exposition artifact for attempt N108

verify_n108_spectral_shield.py
-> experiment artifact
-> supports finite checks and parameter ledger only

N108_verification_log.txt
-> execution output with environment and hash
```

The claim ledger records what each verifier actually checks. A script that exhaustively tests finite cells does not certify the asymptotic theorem unless it emits a separately checkable general certificate.

13.8 How context compilation would improve continuation

At the final high-degree residual target, a context compiler would not replay all 47 pages. It would include:

- the immutable theorem and constructivity kernel;
- N108 and N109 claims actually used by the active route;
- the precise statement that residual compression remains open;
- the algebraic and spectral shield artifacts;
- prior failed compression attempts and their earliest obstruction;
- the candidate XOR/NOT stripping and affine-offset mechanism;
- analogous mechanism records from other branches;
- one or two Hyperon-generated representation changes;
- hashes and source IDs for every included item.

The context manifest would make it clear whether an omitted earlier moment calculation was structurally irrelevant or excluded merely by token budget.

13.9 Example Hyperon intervention at the frontier

Hyperon might observe the following structural pattern:

```
low-degree hiding place eliminated
+ cubic information budget saturated
+ current candidate separates affine structure from AND gates
=> seek a certificate parameterized by multiplicative skeleton rather
    than full circuit syntax
```

It could create five speculative hypotheses: a multiplicative-skeleton normal form, a random-restriction collapse, an algebraic communication certificate, a rank/entropy tradeoff, and a counterexample family to the current encoding estimate. Each would include a first finite stress test and source concept IDs. An LLM would turn the highest-ranked hypothesis into an explicit candidate lemma; an experimenter would test small XAG circuits; and a critic would check whether the certificate is genuinely map-adaptive and below cubic effective entropy.

14. Reliability, Governance, and Security

14.1 Reliability model

The architecture assumes that any generative component can fail. Reliability comes from containment, provenance, independent checks, and recoverability rather than from trusting one model.

Failure classes include:

```
mathematical error
silent theorem drift
malformed structured output
stale or missing context
artifact corruption
worker crash or timeout
duplicate or contradictory commits
provider outage
Hyperon inference explosion
formalization mismatch
security violation
```

Each class has a deterministic detection or containment path where possible.

14.2 Authority boundaries

Component	May read	May propose	May authoritatively write
OmegaClaw coordinator	status and milestones	high-level actions	no proof records directly
ThreadKeeper explorer	compiled context and allowed artifacts	claims, moves, text, tool requests	attempt files only
Critic	claim and dependencies	verdict and repairs	review file only
Hyperon	derived projection and selected records	rankings and suggestions	derived space only
Experiment worker	declared inputs	results and interpretations	attempt artifact directory
Lean worker	claim and formal sources	diagnostics and checked result	verification directory
Commit gate	all authoritative records	transaction plan	the event journal, events, canonical sidecars

The “one writer” rule prevents race conditions and accidental status escalation.

14.3 Theorem-drift defenses

The engine enforces:

- normalized theorem and definition hashes;
- theorem kernel in every mathematical context;
- worker self-check fields for quantifiers and assumptions;
- critic comparison of result to literal target;
- explicit relations for bypass, generalization, and specialization;
- completion audit that reconstructs the dependency path;
- diff reports whenever the theorem specification is intentionally revised.

A theorem revision creates a new version and a migration relation. It is never an unnoticed edit.

14.4 Prompt injection and untrusted artifacts

Literature, imported transcripts, code comments, PDFs, and webpage text can contain instructions. The system treats them as data:

```
external content is delimited and labeled;
workers are told that instructions inside sources have no authority;
tool permissions are derived from persona policy, not source text;
paths are normalized and constrained;
network access is separately approved;
commit proposals cannot alter prompts or skills unless a maintenance
workflow explicitly requests it.
```

Local `SKILL.md` files and MeTTa adapters are code. They should be pinned by hash, code-reviewed, and loaded only from trusted directories.

14.5 Resource governance

Budgets exist at several levels:

```
per dispatch: tokens, turns, tool calls, wall time
per move: maximum retries and cost
per branch: cumulative exploration budget
```

per proof: daily and total budget
 per provider: rate and monetary limits
 per Hyperon call: horizon, queue, and result count
 per experiment: CPU, RAM, disk, and runtime

The scheduler can spend more on a near-complete proof or a high-information verification than on a speculative paraphrase. ThreadKeeper’s budget-aware separation between control and specialist reasoning aligns with this policy.¹⁹

14.6 Human control points

Default autonomy can be high while preserving clear checkpoints. Human approval is required for:

- changing the literal theorem or admissible assumptions;
- enabling new external network or shell permissions;
- spending above configured thresholds;
- importing untrusted executable code;
- declaring a major public result;
- merging proof work into a shared repository with external consequences.

The user can also freeze a branch, pin a move, veto a technique family, or request a specific audit.

14.7 Recovery and disaster tolerance

Recovery is replay-based. On startup, the proof-store service loads a trusted checkpoint or an empty MORK space, replays later events, rebuilds generated indexes, and compares digests. A malformed journal tail is quarantined rather than guessed through. Leases expire and are reaped; late results remain artifacts but require a new lease.

Backups include the journal, checkpoint manifests, prompts, skills, schemas, theorem kernels, and content-addressed objects. Periodic clean replay from an older checkpoint detects snapshot drift. The system should be able to reconstruct a proof project without access to any provider-side chat history.

14.8 Scientific integrity

The system should preserve negative results and uncertainty. It must not rewrite history to make a route look linear. Reports should distinguish:

what was proved
 what was checked computationally
 what was accepted by a critic
 what remains conjectural
 what was bypassed rather than solved
 what failed and what was learned

This is important both for mathematical correctness and for later learning from the proof process.

15. Evaluation Methodology and Metrics

15.1 What should be measured

Completed theorems are the ultimate objective but are too sparse to guide early development. The system needs intermediate metrics that reward genuine research progress and penalize attractive noise.

¹⁹ H. L. Greenblatt et al., *ThreadKeeper: a configurable hybrid OmegaClaw architecture for budget-aware orchestration*, GitHub repository and architecture documents, `threadkeeper` branch, accessed 15 July 2026. The user-supplied repository is a fork under `bgoertzel-sing`.
<https://github.com/bgoertzel-sing/ThreadKeeper>

15.2 Correctness and epistemic discipline

Metric	Definition
theorem-drift rate	fraction of attempts that materially change target semantics without an explicit relation
unsupported-dependency rate	claims that cite a premise not in the ledger or not at the stated status
critic reversal rate	supported claims later rejected by an independent critic
formalization mismatch rate	informal claims whose first Lean statement is materially different
retraction propagation latency	time from refutation to reopening all dependent states
completion false-positive rate	completion candidates rejected by final audit

These metrics test the architecture’s main promise: better state semantics than chat history alone.

15.3 Search productivity

Useful measures include:

verified claims per million tokens
 subgoals closed per worker-hour
 reusable claims created per proof
 branches pruned per experiment
 median attempts before earliest obstruction is identified
 fraction of failures that produce a new obstruction or reusable artifact
 frontier duplication rate across workers
 time from new claim to reuse in another branch

A system that disproves an attractive false lemma quickly may outperform one that produces more prose.

15.4 Context quality

Measure:

- fraction of worker-used facts that came from theorem kernel, MORK structure, PLN, semantic retrieval, or diversity injection;
- retrieval precision judged by worker citations and critic review;
- omission rate for dependencies later found necessary;
- inclusion rate of tainted or status-misrepresented claims;
- context redundancy and token efficiency;
- reproducibility of a packet from its manifest and proof revision;
- sensitivity to removal of reverse indexes, route views, or PLN scores.

15.5 Hyperon creative-suggestion metrics

Track:

- hypotheses generated, elaborated, queued, rejected, and expired;
- novelty relative to existing move and concept neighborhoods;
- critic survival and time-to-falsification;
- subgoals closed or branches pruned;
- reusable claims and Lean-verified descendants;
- mechanism success conditioned on structural state signature;
- human-rated strategic value;

- fraction of human guidance episodes later reproduced by Hyperon;
- rate of speculative suggestions incorrectly presented as facts, which should be zero.

15.6 Human effort

The original motivation includes reducing repetitive human prompting. Measure:

human messages per durable milestone
 manual “continue” or “next move” interventions per 100 cycles
 human corrections of state or theorem status
 human time spent reconstructing context
 fraction of human guidance episodes later anticipated by Hyperon

A successful system should reduce mechanical guidance while preserving high-value mathematical collaboration.

15.7 Cost and latency

Track:

tokens and monetary cost by persona and outcome
 local versus cloud specialist usage
 cost per critic-accepted claim
 cost of duplicate work
 context compilation latency
 Hyperon inference latency
 experiment and Lean queue time

ThreadKeeper’s design thesis can be tested by comparing frequent inexpensive control plus selective specialist use against using a premium model on every cycle.

15.8 Ablation plan

A credible evaluation should compare:

1. **Chat baseline:** one long conversation with manual prompting.
2. **Persistent workspace only:** history, graph, ledger, artifacts; no context optimization.
3. **Context compiler:** deterministic graph and semantic retrieval.
4. **Parallel workers:** shared leased frontier.
5. **Hyperon ranking only:** no creative operators.
6. **Hyperon creative:** suggestions and human-guidance learning enabled.
7. **Critic and experiments:** independent review and computation.
8. **Lean integration:** formal verification for selected claims.

Use both synthetic fixtures with known hidden errors and real open-ended proof efforts. The same theorem states and budgets should be replayed across conditions when possible.

15.9 Benchmark suite

The benchmark should contain several task types:

long elementary proof with many lemmas
 proof requiring a decisive counterexample
 finite combinatorial theorem amenable to exhaustive checking
 analysis/algebra theorem where definitions and quantifiers are delicate
 research-style route with useful bypasses and barrier theorems
 formalization-friendly theorem with a mathlib path
 case study imported from a real LLM interaction

The goal is not to rank models on one-step theorem proving. It is to test persistence, search management, creative strategy formation, and verification over long trajectories.

16. Implementation Roadmap

16.1 Phase 0: import and replay fixture

Use the supplied N105–N109 proof effort as the first realistic fixture. Create theorem, state, move, claim, attempt, route, artifact, bypass, and barrier atoms. Write tests that reject a false overall-completion label.

16.2 Phase 1: journal, artifact store, and in-memory atom adapter

Implement stable IDs, JSONL events, hash chaining, revision checks, content-addressed artifacts, deterministic replay, one explorer, and one context compiler. Use an in-memory MORK-compatible adapter for tests.

Exit criterion: crash, replay, and resume without losing or reclassifying proof state.

16.3 Phase 2: live PeTTa/MORK adapter

Build pinned PeTTa with MORK support. Implement add/remove/match/MM2 through the proof-store service, benchmark ground-prefix queries, and verify that replay produces the expected committed-space digest.

16.4 Phase 3: critics, experiments, and audits

Add independent criticism, experiment manifests and resource limits, taint propagation, theorem-drift checks, milestone reports, and deterministic exports.

16.5 Phase 4: two or three parallel workers

Add atomic leases, fencing tokens, virtual loss, stale-result handling, and model/persona diversity. Measure duplicate work and objective claim yield.

16.6 Phase 5: Hyperon relevance and creativity

Define concept and mechanism ontologies. Add speculative hypothesis atoms, missing-lemma abduction, analogy, concept blending, representation change, obstruction inversion, counterexample generation, and LLM elaboration. Learn from mechanism outcomes and human guidance.

16.7 Phase 6: Lean 4

Pin the toolchain and libraries. Formalize stable claims incrementally, link declaration hashes and dependency versions, and feed classified errors back into the active branch.

16.8 Phase 7: larger-scale learned search

After collecting attempt data, add calibrated best-first values, PUCT, progressive widening, proof-number signals, and transposition statistics. Shard by theorem and introduce asynchronous dispatch only when it improves verified progress per cost.

16.9 First engineering sprint

A practical first sprint delivers:

1. proof specification and workspace creation;
2. journal and artifact store;
3. MORK atom contract and in-memory adapter;
4. replay, digest, and audit;

5. atomic two-move lease test;
6. coordinator and explorer prompts;
7. context manifest;
8. N105–N109 import sketch;
9. one end-to-end dry run with a manually supplied worker result.

16.10 Architectural decision summary

Persistent agent	proof project, not conversation
Semantic graph	MORK-backed PeTTa metagraph
Durability	revisioned append-only journal
Large content	content-addressed artifact store
Mutation	deterministic commit gate only
Parallelism	atomic leases with fencing tokens
Memory selection	MORK + PLN + semantic retrieval + diversity
Creativity	Hyperon speculative hypothesis layer
Workers	bounded ThreadKeeper personas
Verification	critics, reproducible experiments, incremental Lean

Appendix A. MORK Atom and Event Reference

A.1 Search atoms

```
(proof p17 layer committed state s101)
(proof p17 state-kind s101 or)
(proof p17 state-status s101 open)

(proof p17 layer committed move m204)
(proof p17 move-kind m204 reduction)
(proof p17 move-status m204 queued)

(proof p17 out s101 proposes m204 e901)
(proof p17 in m204 proposed-by s101 e901)
(proof p17 out m204 requires s102 e902)
(proof p17 in s102 required-by m204 e902)
```

A.2 Claim and provenance atoms

```
(proof p17 layer committed claim c308)
(proof p17 claim-status c308 provisional)
(proof p17 claim-statement-blob c308 b3_CLAIM)
(proof p17 depends-on c310 c308)

(proof p17 layer committed attempt a411)
(proof p17 attempt-on a411 m204)
(proof p17 attempt-at-state a411 s101)
(proof p17 attempt-via-route a411 r52)
(proof p17 context-manifest a411 ctx411)
(proof p17 produced-claim a411 c308)
```

A.3 Artifact atoms

```
(proof p17 layer committed artifact art92)
(proof p17 artifact-kind art92 python-source)
(proof p17 artifact-blob art92 b3_4VQ7ZJ6)
(proof p17 artifact-media-type art92 text-x-python)
(proof p17 produced-artifact a411 art92)
```

A.4 Speculative Hyperon atoms

```
(proof p17 layer speculative hypothesis h701)
(proof p17 hypothesis-kind h701 representation-change)
(proof p17 targets-subgoal h701 s103)
(proof p17 source-concept h701 concept17)
(proof p17 suggests-mechanism h701 mech27)
```

```
(proof p17 novelty h701 score081)
(proof p17 falsification-test h701 test44)
```

A.5 Event bundle

```
{
  "event_id": "ev-00982",
  "proof_id": "p17",
  "expected_revision": 981,
  "new_revision": 982,
  "actor": "explorer-03",
  "lease": {
    "lease_id": "lease-771",
    "fencing_token": 14
  },
  "operations": [
    {"op": "add", "atom": ["proof", "p17", "claim", "c308"]},
    {"op": "add", "atom": ["proof", "p17", "claim-status", "c308", "provisional"]},
    {"op": "add", "atom": ["proof", "p17", "produced-claim", "a411", "c308"]}
  ],
  "metadata": {
    "attempt_id": "a411",
    "context_id": "ctx411",
    "result_blob": "b3_RESULT"
  }
}
```

A.6 Atomic lease record

```
(proof p17 lease-active m204 lease771)
(proof p17 lease-worker lease771 explorer03)
(proof p17 lease-expiry lease771 t20260715T120000Z)
(proof p17 lease-token lease771 14)
(proof p17 lease-route lease771 r52)
(proof p17 move-fence m204 14)
```

A.7 Illustrative MM2 reverse index

The exact syntax must be tested against the pinned MORK release.

```
(exec (index 0)
      (, (proof $p out $src $rel $dst $edge))
      (, (proof $p in $dst $rel $src $edge)))
```

A.8 Snapshot manifest

```
{
  "proof_id": "p17",
  "revision": 982,
  "event_tail_hash": "...",
  "committed_atom_digest": "...",
  "generated_index_digest": "...",
  "artifact_manifest_root": "...",
  "petta_revision": "...",
  "mork_revision": "...",
  "maintenance_rule_hashes": ["..."]
}
```

Appendix B. Structured Worker, Context, Event, and Suggestion Contracts

The companion bundle contains complete JSON Schema Draft 2020-12 files:

```
schemas/proof-spec.schema.json
schemas/proof-event.schema.json
schemas/worker-result.schema.json
schemas/context-manifest.schema.json
schemas/experiment-spec.schema.json
schemas/hyperon-suggestion.schema.json
```

schemas/claim.schema.json
schemas/attempt.schema.json

B.1 Worker result

The worker result binds the mathematical proposal to proof, state, move, route, lease, fencing token, and base revision. It distinguishes disposition from relation to the literal target and contains proposed claims, states, moves, alternatives, artifacts, gaps, and assumption changes.

B.2 Context manifest

Each source record contains source ID and version, retrieval channel, inclusion reason, status, score, token count, and optional exclusion reason. The manifest also records the packet hash, proof revision, lease, and fencing token.

B.3 Hyperon suggestion

A suggestion contains hypothesis ID, kind, target state, mechanism, candidate lemma, source and prerequisite IDs, expected consequences, falsification test, abductive strength, novelty, relevance, cost, risk, and speculative lifecycle status.

B.4 Event transaction

An event contains expected and new revisions, actor, timestamp, optional lease reference, ordered atom additions/removals, and metadata. The journal implementation adds the previous-event hash and event hash.

B.5 Experiment and claim records

Experiment specifications state the question, hypotheses, domain, seed policy, outcomes, inferential limits, and resources before code runs. A claim record stores exact statement, protocol status, dependencies, source attempt, critics, Lean runs, and taint sources.

Appendix C. Prompt Templates and Skill Specifications

The companion bundle contains the normative files:

prompts/omega-mathematician.md
prompts/theorem-importer.md
prompts/milestone-reporter.md

skills/proof-orchestrator/SKILL.md
skills/proof-mork-store/SKILL.md
skills/proof-context/SKILL.md
skills/proof-step/SKILL.md
skills/proof-critic/SKILL.md
skills/proof-supermind/SKILL.md
skills/proof-hyperon-creative/SKILL.md
skills/proof-experiment/SKILL.md
skills/lean4-formalizer/SKILL.md
skills/proof-audit/SKILL.md

C.1 Coordinator policy excerpt

The authoritative record is the proof workspace, not chat history.
If one move is clearly best, execute it.
If several moves are viable, execute one and queue the others.
After two non-progress attempts, invoke five-mechanism expansion.
Never silently change theorem, assumptions, quantifiers, or constructivity requirements.

A bypass or stronger substitute is progress but not literal completion without an explicit implication chain.

C.2 Worker result discipline

Every worker writes a structured result with disposition, relation to target, new claims and dependencies, new states and moves, closed goals, remaining gaps, obstructions, alternatives, artifacts, and assumption changes. Workers do not mutate MORK.

C.3 Hyperon creative discipline

Every creative suggestion is a speculative hypothesis with source IDs, mechanism, candidate lemma, prerequisites, consequences, novelty, abductive support, falsification test, and status. The LLM elaborates it; the commit gate does not treat it as proof.

C.4 ThreadKeeper digest discipline

The full result goes to the attempt directory. The returned digest is a single line containing attempt ID, disposition, and result path.

Appendix D. Configuration Example

```
schema_version: 1
workspace_root: ./proof-workspace
```

```
store:
  semantic_backend: mork
  durability_backend: jsonl
  one_process_per_proof: true
  checkpoint_every_events: 100
  event_hash_chain: true
  maintenance_iteration_limit: 64
```

```
artifacts:
  digest: sha256
  large_object_backend: local
```

```
scheduler:
  policy: best_first_and_or
  lease_ttl_seconds: 1800
  virtual_loss: 0.15
  stagnation_attempts_before_supermind: 2
```

```
context:
  token_budget: 60000
  structural_weight: 4.0
  ancestry_weight: 3.0
  semantic_weight: 2.5
  verification_weight: 2.0
  pln_weight: 1.3
  novelty_weight: 1.0
  redundancy_penalty: 1.5
```

```
hyperon:
  enabled: true
  write_layer: speculative
  creative_operators:
    - missing-lemma-abduction
    - analogy-transfer
    - concept-blending
    - representation-change
    - obstruction-inversion
```

```

- counterexample
- random-deterministic-hybrid
max_hypotheses_per_run: 12

threadkeeper:
  enabled: true
  full_results_to_files: true
  digest_max_characters: 1200

lean4:
  enabled: false
  project_path: ./lean
  command: lake env lean

security:
  allow_paths_under_workspace_only: true
  worker_network_default: false
  shell_wall_seconds: 900
  shell_memory_mb: 8192

```

References

The repository and documentation references below are implementation context, not dependencies of the mathematical claims in any proof project.

1. ASI Alliance. *OmegaClaw-Core*. GitHub repository and project documentation. Accessed 15 July 2026.
2. TrueAGI. *PeTTa*. GitHub repository and MORK-space documentation. Accessed 15 July 2026.
3. TrueAGI. *MORK*. GitHub repository and wiki pages on data encoding, space operations, and MM2 reachability. Accessed 15 July 2026.
4. Patham9. *mork_ffi*. Public Rust FFI source. Accessed 15 July 2026.
5. B. Goertzel-Sing fork and H. L. Greenblatt et al. *ThreadKeeper*. GitHub repository and subagent-design documentation. Accessed 15 July
6. OpenCog Hyperon. Project overview and experimental repository. Accessed 15 July 2026.
7. TrueAGI. *Probabilistic Logic Networks: MeTTa-native PLN for Hyperon*. Accessed 15 July 2026.
8. Lean Project and Mathlib Community. *Lean 4*, *Lake*, and *mathlib4*.
9. *Proof Effort Continuation*. 47-page PDF supplied with this design, especially pp. 27–46 (N105–N109 and artifact lists).

Companion implementation bundle. The source package accompanying this document contains the MORK schema, illustrative MM2 rules, JSON contracts, OmegaClaw wrappers, ThreadKeeper personas, skill files, Python reference service, tests, and import/replay examples.